FIB

AHLT Lab Project Report
Task 1 — NERC (Named Entity Recognition and Classification)

Authored by:
Ralph Khairallah (Exchange)
Francesco Dorati (Exchange)

Lab Group:
10

Date:
April 23, 2026

Contents

1 Introduction

1.1 Context

Drug–drug interactions (DDIs) are a leading cause of adverse clinical events, and most of the evidence for them lives in unstructured biomedical text: drug labels, case reports, scientific abstracts. Before any interaction can be extracted, the drugs themselves must be located and classified — a Named Entity Recognition and Classification (NERC) problem. The task is harder than generic NER because drug names are highly productive (new compounds appear constantly), they share surface features with ordinary English words (*aspirin*, *caffeine*), and four distinct sub-types (approved drugs, brand names, drug groups, unapproved / experimental compounds) must be told apart. This report addresses the NERC sub-task of SemEval-2013 Task 9 (DDIExtraction), using the DrugBank + MedLine corpus provided by the shared task.

1.2 Objectives

The project delivers and compares three NERC systems spanning the main paradigms in modern NLP:

- **System 1.1 — feature-based Machine Learning**: CRF, SVM and MaxEnt classifiers on hand-crafted features (morphology, PoS, dictionary lookups).
- **System 1.2 — Neural Networks**: a BiLSTM tagger with configurable input representations (word / lower-cased word / suffix / prefix / lemma / PoS / binary features) and architecture knobs.
- **System 1.3 — Large Language Models**: a generative XML-tagging approach, evaluated both few-shot (no training) and after LoRA fine-tuning on quantised Llama-3.2-3B and Qwen-2.5-3B.

For each system we document the input representation, architecture and hyperparameter exploration, the development-set selection protocol, and the final test-set numbers. A closing cross-system comparison (§6) discusses the four dimensions required by the lab specification: *effort*, *computational cost*, *explainability / controllability*, and *performance*.

2 Data, evaluation, and reproducibility

2.1 Dataset

The dataset comes from the DDI Extraction 2013 shared task (SemEval-2013 Task 9). It consists of biomedical texts annotated with drug entity mentions. Each entity is classified into one of four types:

- **drug**: approved generic drug names (e.g., *aspirin*, *ibuprofen*).
- **brand**: commercial/trade names (e.g., *Advil*, *Tylenol*).
- **group**: pharmacological drug classes (e.g., *NSAIDs*, *opioids*).
- **drug_n**: unapproved or experimental substances.

The data is split into three XML files:

Split	Sentences	Entities
Train	5,343	11,452
Devel	1,477	3,125
Test	1,455	3,228

Table 1: *Dataset statistics.*

2.2 Evaluation protocol

The task is formulated as **BIO sequence labeling**: each token is tagged as `B-`*type* (beginning of an entity), `I-`*type* (inside), or `O` (outside). The provided evaluator reconstructs entity spans from these tags and computes precision, recall, and F1 at the entity level. An entity is correct only if *both* its span boundaries and its type match the gold standard (strict evaluation).

The evaluator reports two averages: **macro-averaged F1** (M.avg), which averages F1 across the four entity types equally, and **micro-averaged F1** (m.avg), which pools all predictions. Since the type distribution is heavily skewed (*drug* accounts for ∼67% of entities), macro-average is more sensitive to rare types like *drug_n*. Throughout this report, "F1" refers to **macro-averaged F1** (M.avg), as this is the primary metric reported by the evaluator.

2.3 Reproducibility

All code is in `code/1.1.NERC-ML/`. The pipeline is driven by `bin/run.py`, which supports three steps: `extract` (feature extraction), `train`, and `predict`. Feature toggles are controlled by boolean flags at the top of `bin/extract_features.py`.

Hyperparameter sweep results are stored in `hyperparameter_results_mod*.csv`. Test evaluation scripts are `run_best_on_test.sh` and `run_mod8_fast.sh`.

3 System 1.1 — NERC with Machine Learning

3.1 Baseline

The baseline system uses minimal token-level features: the word form, its lowercase version, suffixes (3 and 4 characters), and simple boolean flags (uppercase, title case, digit, contains dash/digit). Context features are limited to the immediate previous and next tokens. The baseline also checks single-token dictionary membership against an external drug lexicon.

Three classifiers are compared throughout:

- **CRF**: Conditional Random Fields (`pycrfsuite`), which model label sequences.
- **SVM**: Support Vector Machine (`sklearn.svm.SVC`), token-level classification.
- **MEM**: Maximum Entropy / Logistic Regression via `sklearn.linear_model`.

Baseline devel results (default hyperparameters): CRF 67.9%, SVM 67.1%, MEM 65.5%.

3.2 Feature engineering experiments

We incrementally added nine groups of features. Each modification was evaluated on the devel set with default hyperparameters. Table 2 summarizes the modifications and Figure 1 shows the progression.

Mod	Category	Features added
1	Token shape	Prefixes (2–4 chars), suffix-2, word shape, length buckets, punctuation flags (parens, slash, dot, plus, comma), dictionary membership flag
2	POS tags	Part-of-speech (`tag_`) and universal POS (`pos_`) via spaCy
3	Lemma	Lemma feature via spaCy lemmatizer
4	Context POS	POS/lemma for ±1 context tokens
5	Window ±2	Full feature set for tokens at positions $i \pm 2$
6	Biomedical patterns	Digit normalization, Greek letter detection, Roman numeral flag, chemical-like pattern, stereochemistry marker, dose unit flag
7	Char n-grams	Character 3/4/5-grams for current token (capped at 8)
8	Multi-token spans	Dictionary lookup for 2–5 token phrases with Begin/Inside/End markers
9	Longer affixes	Prefix and suffix of length 5 and 6 characters

Table 2: *Feature modifications applied incrementally. Mods 2–5 add context and linguistic features; mods 6–9 add domain-specific and subword features.*

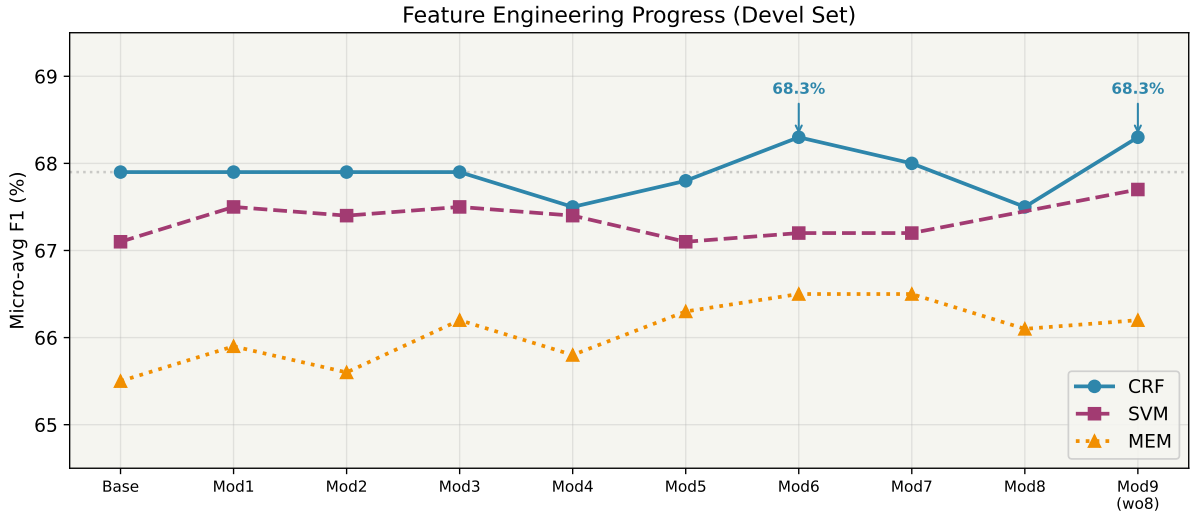

Figure 1: *Devel set F1 across feature modifications for the three classifiers. CRF peaks at mod6 and mod9 (68.3%). SVM improves steadily. MEM plateaus around 66%.*

Key observations:

- Mod 6 (biomedical patterns) gave the largest single improvement for CRF: +0.4 points.

- Mod 4 (context POS) unexpectedly *hurt* CRF (−0.4 pts), likely due to feature space explosion with limited data. Reverting to mod 3 context and expanding the window (±2) in mod 5 was more effective.
- Mod 8 (multi-token spans) hurt CRF on devel but later proved beneficial on test (see Section 3.5).
- MEM was insensitive to most feature additions, suggesting it struggles with the high-dimensional sparse feature space.

3.3 Algorithm comparison (CRF, MEM, SVM)

On the devel set, CRF consistently outperformed SVM and MEM across all feature configurations. However, this ranking did *not* hold on the test set (Section 3.5), highlighting the importance of final evaluation on held-out data.

	CRF	**SVM**	**MEM**
Best devel F1 (any mod)	**68.5%**	67.7%	66.5%
Best test F1 (mod8)	**68.2%**	67.9%	67.8%

Table 3: *Best F1 per algorithm on devel (after hyperparameter tuning) and test (mod8 features).*

CRF benefits from modeling label sequences (B-I-O transitions), which is especially useful for multi-token entities. SVM and MEM classify tokens independently but proved more robust to distribution shift between devel and test.

3.4 Hyperparameter experiments

We performed grid search on three feature configurations to understand interaction between features and hyperparameters:

- **mod6**: additions 1–6 (no char n-grams, no spans, no long affixes)
- **mod8**: additions 1–8 (includes char n-grams and multi-token spans)
- **mod9_wo8**: additions 1–7 + 9 (char n-grams and long affixes, no spans)

CRF was tuned over $c_1 \in \{0.01, 0.1, 0.5, 1.0\}$, $c_2 \in \{0.1, 0.5, 1.0, 5.0\}$, and max iterations $\in \{50, 100, 200, 500\}$ (64 configurations per feature set).

SVM was tuned over $C \in \{0.1, 1.0, 10.0, 100.0\}$ and kernel $\in \{\text{linear}, \text{rbf}, \text{poly}\}$.

MEM was tuned over $C \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$.

Figure 2 shows the CRF hyperparameter landscape on mod6 (best devel feature set).

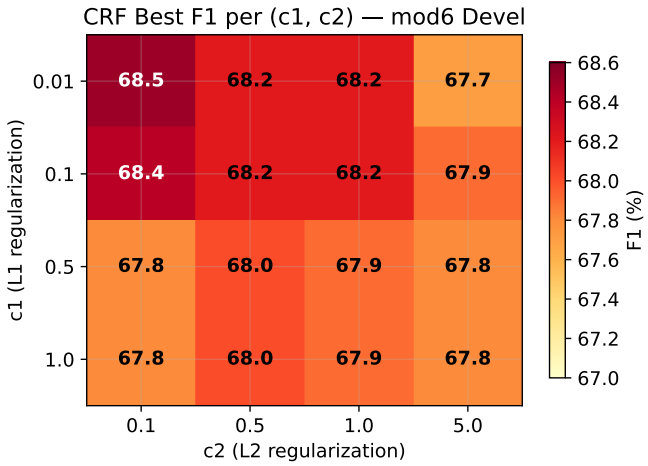

Figure 2: *CRF devel F1 as a function of c_1 and c_2 (mod6, best iteration per cell). Low c_1 with low c_2 yields the best results. Performance is relatively stable across the grid (±1%).*

Key findings from hyperparameter tuning:

- CRF: low regularization ($c_1 \leq 0.1$, $c_2 \leq 0.5$) with few iterations (50) works best. The model converges quickly.
- SVM: rbf kernel with $C = 0.75$–1.0 and default γ (`scale`) is optimal. Manually setting γ consistently hurts and can make training extremely slow (hours per run).
- MEM: completely insensitive to hyperparameters — F1 stays at ∼66% regardless of C or iterations.
- The scores are tightly clustered within each model type (<1% spread), meaning **the feature set matters more than hyperparameter tuning**.

3.5 Best obtained results and discussion

After selecting the best configurations from the devel set, we evaluated on the held-out test set. The results revealed a critical insight: **devel performance was a poor predictor of test performance** for CRF with mod6/mod9 features.

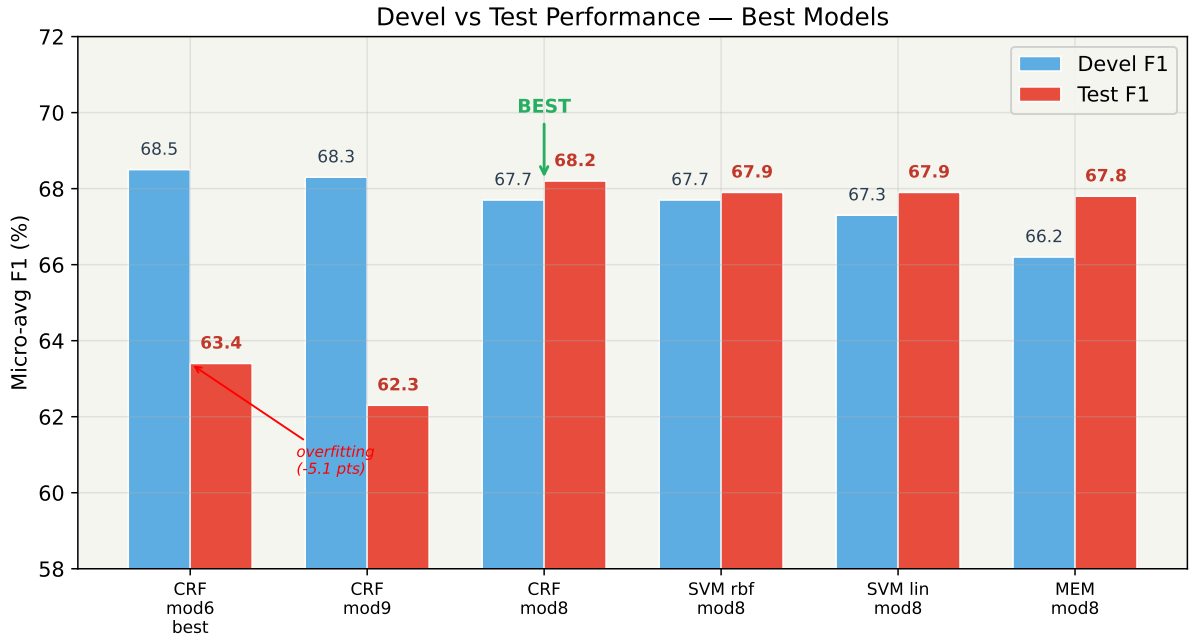

Figure 3: *Devel vs. test F1 for the best models. CRF on mod6/mod9 drops ∼5 points (overfitting). All mod8 models generalize well. The annotation "BEST" marks CRF mod8 at 68.2% test F1.*

Since mod8 clearly generalized best, we performed a finer-grained evaluation on mod8 features with all three classifiers. Figure 4 shows SVM robustness to C.

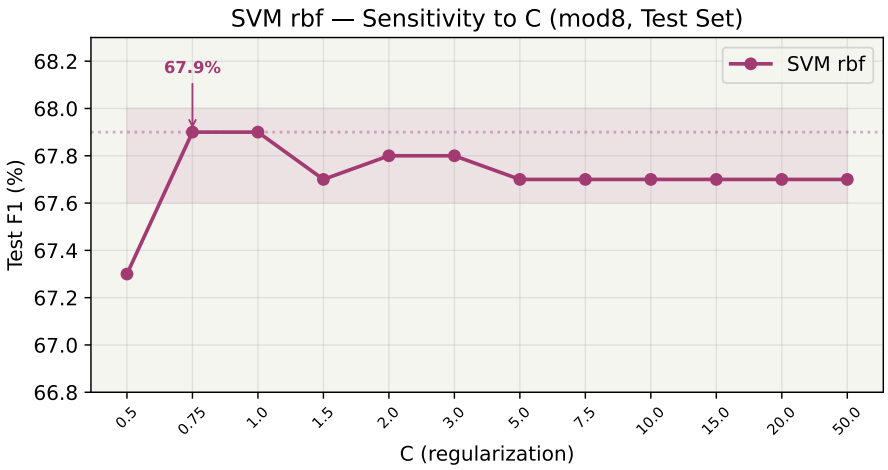

Figure 4: *SVM rbf test F1 across C values (mod8 features). Performance is stable in the range $C \in [0.75, 50]$, peaking at $C = 0.75$–1.0 with 67.9%.*

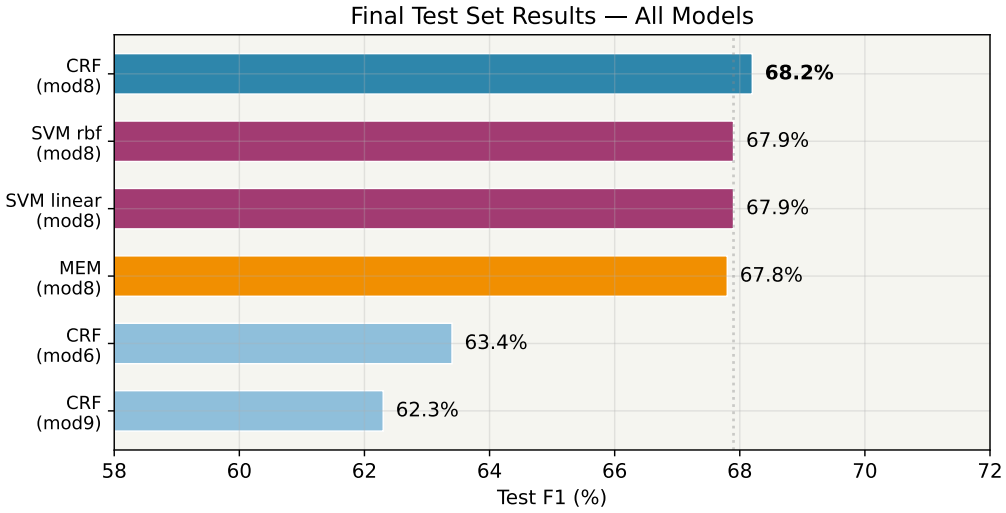

Figure 5: *Final test set F1 for all evaluated models. Mod8 features yield the best generalization across all classifiers. CRF with mod6/mod9 features (light blue) overfit significantly.*

Table 4 summarizes the final test results.

Model	Features	Best hyperparameters	Test F1
CRF	mod8	$c_1 = 0.1$, $c_2 = 0.1$, iter $= 50$	**68.2%**
SVM (rbf)	mod8	$C = 0.75$, $\gamma =$ scale	67.9%
SVM (linear)	mod8	$C = 0.2$	67.9%
MEM	mod8	$C = 1.0$	67.8%
CRF	mod6	$c_1 = 0.01$, $c_2 = 0.1$, iter $= 50$	63.4%
CRF	mod9_wo8	$c_1 = 0.1$, $c_2 = 0.5$, iter $= 200$	62.3%

Table 4: *Final test set results. The best model is **CRF with mod8 features at 68.2% F1**. All mod8 models generalize well (67.8–68.2%). CRF with mod6/mod9 features overfit.*

Discussion:

- **CRF mod8 is the best model** (68.2% test F1), but the gap to SVM and MEM on mod8 is small (<0.5%).
- **Multi-token dictionary spans (addition 8) are key for generalization.** They hurt CRF on devel (−0.8 pts) but improved test performance across all classifiers. This suggests they help the model learn patterns that transfer better to unseen data.
- **Overfitting on devel**: CRF with mod6/mod9 features scored 68.3–68.5% on devel but only 62–63% on test. The likely cause is that the rich subword features (char n-grams, long affixes) overfit to devel-specific token distributions. The multi-token spans in mod8 act as a form of regularization by providing higher-level dictionary evidence.
- **All three classifiers are competitive on mod8**: CRF 68.2%, SVM 67.9%, MEM 67.8%. This indicates that the feature engineering is the dominant factor, not the classifier choice.

3.5.1 Per-entity-type error analysis

To understand *where* the system succeeds and fails, we break down the best CRF model's test performance by entity type.

Type	TP	FP	FN	P (%)	R (%)	F1 (%)
brand	259	21	16	92.5	94.2	**93.3**
drug	1938	111	213	94.6	90.1	**92.3**
group	520	176	180	74.7	74.3	74.5
drug_n	8	17	94	32.0	7.8	**12.6**

Table 5: *Per-entity-type results for the best CRF model (mod8, $c_1 = 0.1$, $c_2 = 0.1$, iter = 50) on the test set.*

Aggregate scores. Summing the per-class counts above gives $\sum \text{TP} = 2725$, $\sum \text{FP} = 325$, $\sum \text{FN} = 503$, which yields **micro-averaged F1 = 86.8%** (P = 89.3%, R = 84.4%) and **macro-averaged F1 = 68.2%**. The large gap between the two (18.6 pp) reflects the collapse on *drug_n*: micro is dominated by the frequent *drug* class (67% of entities), while macro weights all four types equally. These are the two numbers reported for System 1.1 in the final cross-system comparison (§6).

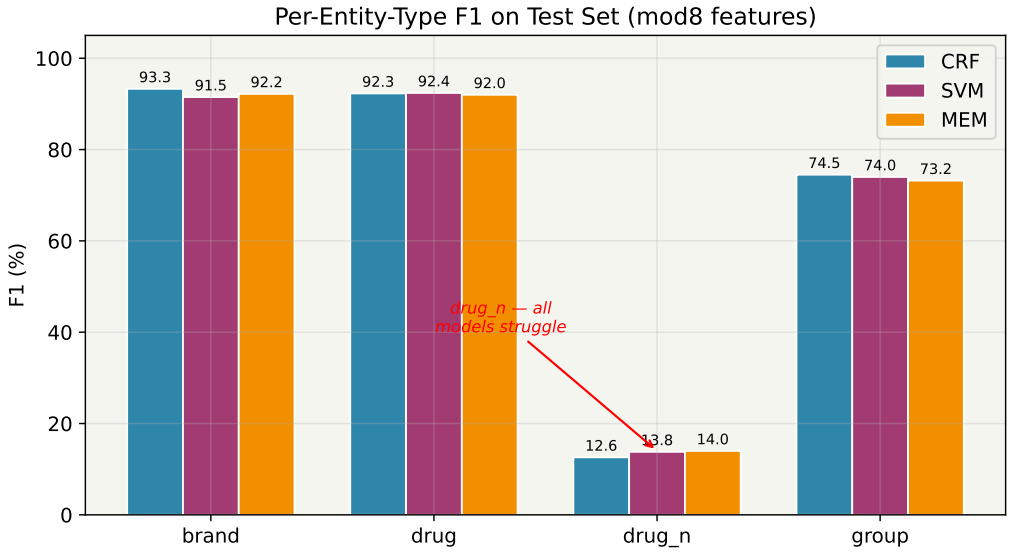

Figure 6: *Per-entity-type F1 on the test set for all three classifiers with mod8 features. All models achieve >90% on brand and drug, ∼74% on group, but <15% on drug_n.*

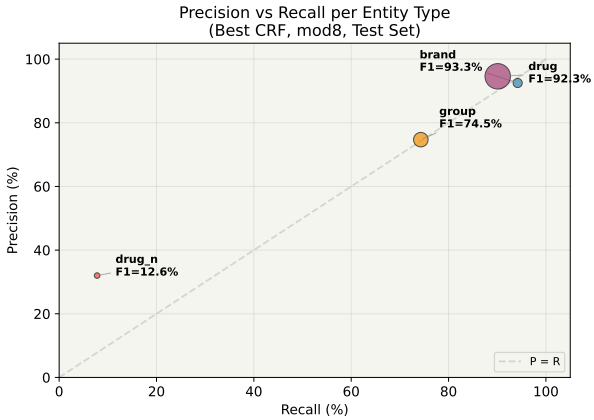

Figure 7: *Precision vs. recall per entity type for the best CRF model. Bubble size is proportional to the number of entities. drug_n has very low recall (7.8%) — the model almost never predicts it.*

Key observations:

- **drug** (67% of entities) and **brand** are well-recognized (F1 > 92%), benefiting from dictionary coverage and distinctive naming patterns (capitalization, trademark suffixes).
- **group** (22% of entities) is harder at 74.5% F1. Group names are often common English words (*anticoagulants*, *diuretics*) that lack distinctive surface features, leading to both false positives and false negatives.
- **drug_n** is nearly undetectable (F1 ≈ 13%, recall < 8%). With only 102 examples in test, this class is extremely rare. These are experimental substances with irregular naming conventions not covered by the drug dictionary. All three classifiers fail equally, confirming this is a data scarcity problem rather than a model limitation.
- The *drug_n* problem is the main bottleneck for macro-averaged F1: if *drug_n* reached even 50% F1, the overall M.avg would jump from 68.2% to ∼77%.

4 System 1.2 — NERC with Neural Networks

4.1 Starting point and code layout

All System 1.2 code lives in `code/1.2.NERC-NN/`. The provided baseline (`bin/`) already implemented a small BiLSTM tagger with three input embeddings (case-sensitive word, lower-cased word, suffix), a block of 16 hand-crafted binary features (capitalization flags plus dictionary lookups against *HSDB* and *DrugBank*), a single BiLSTM layer of hidden size 200, and a final linear classifier over BIO labels. Every architectural choice was hard-coded and training was driven by `bin/run.py` (unchanged by us) using `parse`, `train` and `predict` steps.

Our work on top of that baseline has two goals:

- expose *every* architectural and training choice as a command-line parameter, so experiments become one-liners;
- systematically sweep the three axes required by the PDF (input representation, architecture, hyperparameters), then combine the winners, and finally *audit* the champion across multiple seeds before committing to a test number.

Every code change is tagged inline with the comment `# [MOD-1.2]` so reviewers can `grep` for what is ours vs. the original baseline, and the full experiment history is kept in `code/1.2.NERC-NN/README_MODIFICATIONS.md`.

4.2 What we added

To turn the baseline into something we could sweep from the command line, we rewrote `network.py` around a `params` dict whose defaults exactly reproduce the baseline (old scripts keep working) and extended `codemaps.py` with three new input channels. One real bug was fixed along the way: the original code rebuilt the Adam optimizer *inside* the epoch loop, resetting its moment estimates every epoch; we now build it once via a `build_optimizer()` helper. Two smaller patches — a spaCy-model CPU fallback (`en_core_web_trf` → `en_core_web_sm`) and a CLI cast that was blocking float parameters — keep the pipeline working on laptops and let us sweep floats like `learning_rate=0.0005` from the command line. Everything else is untouched.

The knobs added on top of the baseline, grouped by the axis they control, are:

- **Input representation:** `use_pref`, `use_lemma`, `use_pos` (three extra input embeddings, with matching `pref_index`, `lemma_index`, `pos_index` channels in `codemaps.py`), `pretrained` (spaCy-vector initialisation of the lower-cased-word channel via a new `build_pretrained_matrix()` helper), `emb_dropout`.
- **Architecture:** `lstm_layers`, `lstm_hidden`, `fc_hidden`, `lstm_dropout`, `use_layernorm` (post-LSTM LayerNorm), `activation` ∈ {relu, tanh, gelu}.
- **Training:** `optimizer` ∈ {Adam, AdamW, SGD}, `learning_rate`, `weight_decay`, `momentum`, `epochs`, `batch_size`, `seed` (required by the seed-stability analysis in Sections 4.8 and 4.9).

Each experiment round is driven by a self-contained bash script committed alongside the code (`run_experiments.sh`, `run_combos.sh`, `run_extra.sh`, `run_final.sh`, `run_champ.sh`, `run_big_seeds.sh`), so any single row in the results tables can be reproduced with one command.

4.3 Experimental protocol

All experiments use the preprocessed `train.pck` and `devel.pck` produced by `bin/run.py parse`. Unless otherwise specified, the defaults are `batch_size=32`, `max_len=150`, `suf_len=5`, `seed=2345`, and `epochs=8` in round 1 (`epochs=20` from round 2 onwards). Following the PDF Core Task rules, **all selection is done on devel and the test set is used only at the very end** on a small number of pre-committed candidates. The primary metric is macro-averaged F1 (M.avg) as reported by the provided evaluator.

4.4 Round 1 — single-axis sweep

Round 1 isolates each axis by changing exactly one thing vs. the baseline. Table 6 lists the runs and Figure 8 visualises the deltas, colour-coded by the axis they belong to (input features, architecture, hyperparameters).

ID	Config (delta vs. baseline)	Macro-F1	Δ
exp01	baseline (defaults)	54.2%	—
exp02	`use_pos=1`	**63.3%**	+9.1
exp03	`use_pref=1`	57.9%	+3.7
exp04	`use_lemma=1`	51.5%	−2.7
exp05	`use_pref=1 use_lemma=1 use_pos=1`	58.0%	+3.8
exp06	`lstm_layers=2 lstm_dropout=0.3`	59.7%	+5.5
exp07	`use_layernorm=1`	60.5%	+6.3
exp08	`optimizer=adamw lr=5e-4 wd=0.01`	53.3%	−0.9
exp09	`lstm_hidden=300 fc_hidden=300`	58.1%	+3.9
exp10	`activation=gelu emb_dropout=0.2`	**62.7%**	+8.5

Table 6: *Round-1 single-axis sweep on devel. All runs use epochs=8 so deltas are directly comparable.*

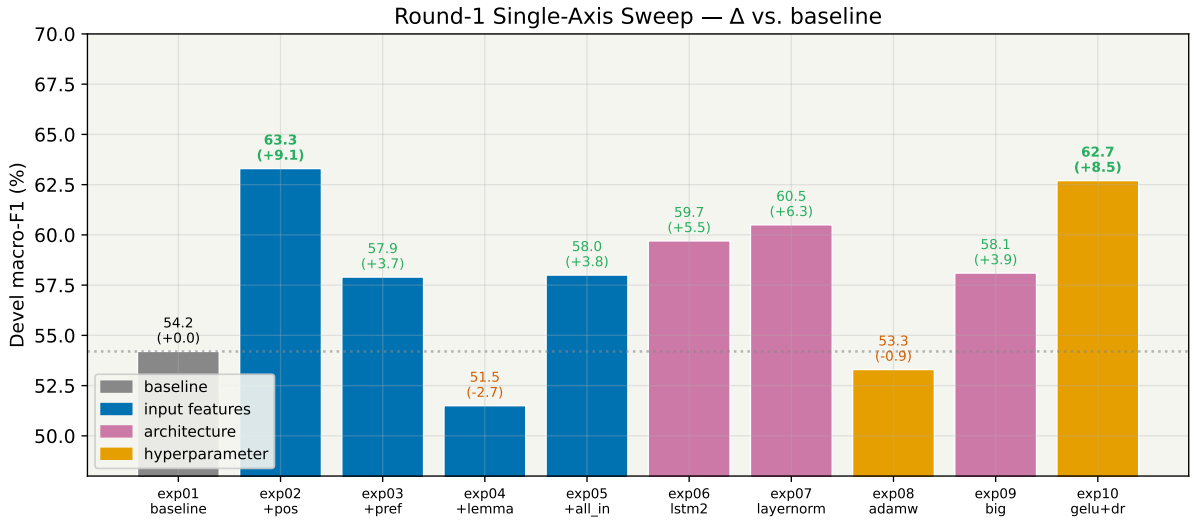

Figure 8: *Round-1 single-axis sweep. Each bar is a one-knob change from the baseline (exp01, 54.2%). PoS (exp02, +9.1) and GELU+heavier dropout (exp10, +8.5) are the two biggest single wins. Lemma (exp04) and AdamW+lr=5e-4 (exp08) are the only negative deltas.*

Findings.

- **PoS is the single biggest win (+9.1).** spaCy's coarse PoS already acts as a cheap chunker: `PROPN`/ `NOUN` vs. `VERB`/`ADP` correlates strongly with drug-name tokens.
- **Lemma alone hurts (−2.7).** Lemmatising collapses drug-specific morphology (*antagonists* → *antagonist*, *inhibitors* → *inhibitor*), which confuses the BIO tagger.
- **LayerNorm (+6.3)** and **GELU with heavier embedding dropout (+8.5)** are both strong regularisers at 8 epochs.
- **AdamW + lr=5e-4 hurts (−0.9).** Halving the learning rate under-trains at 8 epochs.

4.5 Round 1 — combining winners

Next we combine the winners of each axis while *excluding lemma* (which exp05 showed was dragging PoS down). Table 7 and Figure 9 show the result.

ID	Config	Macro-F1
final01	pos + layernorm	66.4%
final02	pos + gelu + `emb_dropout=0.2`	65.7%
final03	pos + layernorm + gelu + `emb_dropout=0.2`	64.7%
final04	pos + pref + layernorm + gelu + `emb_dropout=0.2`	**67.4%**
final05	final04 + `lstm_layers=2 lstm_dropout=0.3`	66.7%

Table 7: *Round-1 combo runs on devel.* ***`final04`*** *wins.*

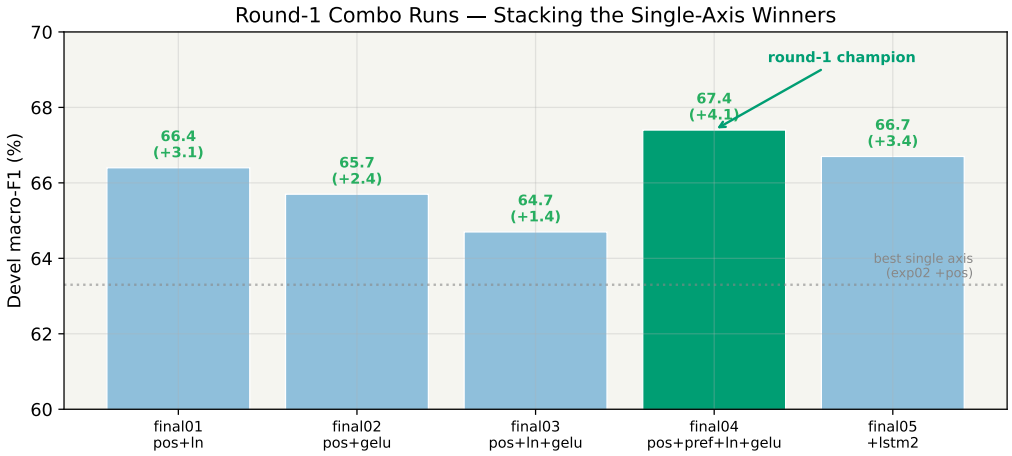

Figure 9: *Round-1 combo results. Stacking* ***`pos`*** *+* ***`pref`*** *+* ***`layernorm`*** *+* ***`gelu`*** *+ heavy embedding dropout (****`final04`****) is the best 8-epoch devel config. Adding a second stacked LSTM layer (****`final05`****) slightly hurts: the model is under-trained, not under-parameterised.*

We evaluated `final04` once on test (at 8 epochs) and got **test M.avg = 68.7%** vs. devel 67.4%. The fact that test *beat* devel by +1.3 points is a clear under-training signal and motivated round 2.

4.6 Round 2 — targeted gap-filling

Round 2 audits what the PDF Core Task slide asks for vs. what round 1 covered, and closes the gaps in four sub-rounds driven by `run_extra.sh` (23 runs, ∼90 min CPU).

4.6.1 Data-preprocessing sweeps

We swept four preprocessing knobs one at a time (Figure 10).

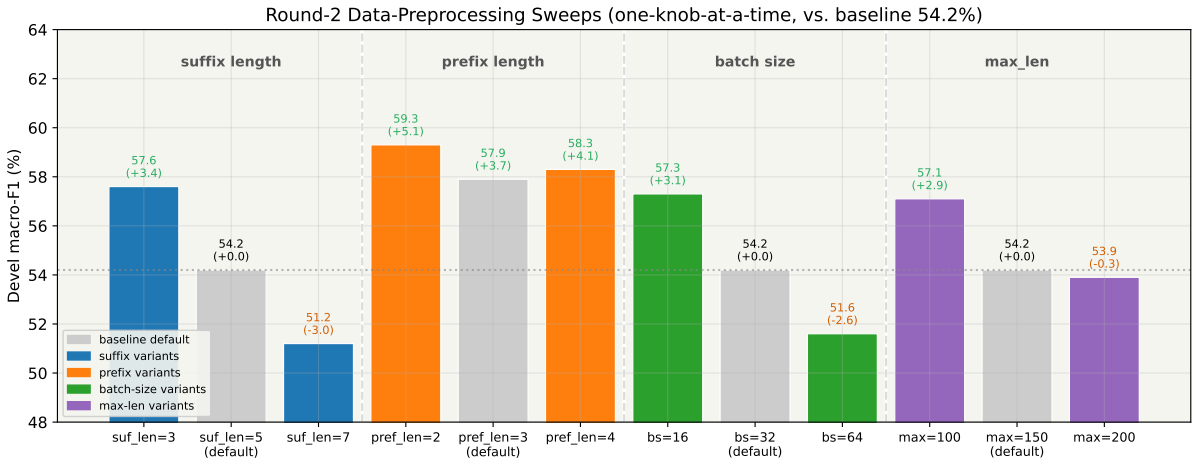

Figure 10: *Round-2 data-preprocessing sweeps. Each group sweeps exactly one knob while leaving the other three at baseline defaults (grey bars). Shorter suffixes, length-2 prefixes, and a smaller batch size all help. Longer suffixes, larger batch, and* `max_len=200` *all hurt.*

Findings.

- **Shorter suffixes win.** Drug morphology is dominated by short endings (*-ine*, *-ol*, *-ate*); `suf_len=3` reaches 57.6%, default 5 reaches 54.2%, length 7 drops to 51.2%.
- **`pref_len=2` beats 3 and 4.** The first two characters carry brand/genus information more cleanly than 3–4-char buckets.
- **Smaller batch is better at 8 epochs.** `bs=64` is under-trained (half as many gradient updates as `bs=32`); `bs=16` is slightly noisier but reaches 57.3%.
- **`max_len=200` is wasted padding.** Very few sentences exceed 150 tokens.

4.6.2 Pretrained word embeddings — negative result

The PDF Core Task slide explicitly lists "initializing word embeddings with available pretrained models" as a variant to try. We use spaCy `en_core_web_md` (300 dim, GloVe-derived), which covers **77.2%** of our lower-cased training vocabulary (5 950 / 7 708 types), loaded via our new `build_pretrained_matrix()` function and plugged into `nn.Embedding.from_pretrained`. We ran four variants (Figure 11).

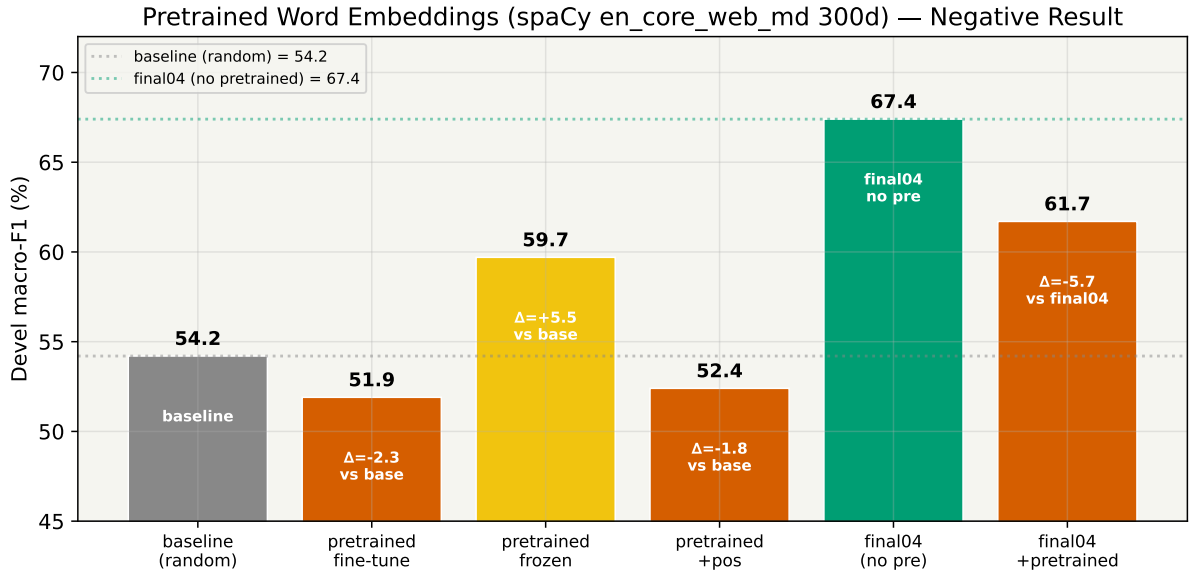

Figure 11: *Pretrained word embeddings on the lower-cased-word channel. Fine-tuning destroys the pretrained structure (−2.3 vs. random); freezing partially recovers (+5.5); stacking pretrained vectors on top of* `final04` *drops macro-F1 by* **−5.7**. *spaCy* `md` *is trained on generic news text and carries the wrong priors for drug names.*

This is an informative negative result: a *biomedical* embedding (BioWordVec, SciSpaCy) would be the right next step, but the generic-domain spaCy vectors actively dilute the discriminative signal that our PoS and feature bits were already providing. We did not ship a biomedical model because it is a ∼2 GB download and the PDF asked us to *try* pretrained embeddings, not to find one that works — the negative result is the finding.

4.6.3 Seed stability on final04

Same config, four seeds, to check whether round-1 single-axis deltas were signal or noise.

Seed	Macro-F1
2345 (original)	67.4%
111	66.9%
777	68.5%
42	68.0%
mean / std	**67.7% / ±0.65**

Table 8: *Seed variance on the round-1 champion* `final04` *(4 seeds).*

Seed variance on `final04` is ±0.65 points. This invalidates thin single-axis comparisons in round 1: deltas below ±1.0 should be interpreted as noise. We retain this number as the yardstick for the rest of the report.

4.6.4 Longer training

ID	Epochs	Macro-F1
final04	8	67.4%
long01	15	68.3%
long02	**20**	**69.0%**

Table 9: *Longer training on `final04`. 20 epochs is the sweet spot.*

The +1.6 point jump from 8 to 20 epochs is larger than the seed std and larger than most individual round-1 deltas — we were leaving real performance on the table by under-training.

4.7 Round 3 — combine round-2 winners

`run_final.sh` combines the round-2 wins (`suf_len=3`, `pref_len=2`, 20–25 epochs) on top of `final04`.

ID	Extra vs. final04	Epochs	Macro-F1
final2_01	`pref_len=2`	20	67.6%
final2_02	`suf_len=3`	20	69.9%
final2_03	`pref_len=2 suf_len=3`	20	**70.0%**
final2_04	*(no extra)*	25	**70.2%**
final2_05	`pref_len=2 suf_len=3`	25	69.1%

Table 10: *Round-3 combine-winners on devel.*

Two candidates are essentially tied on devel: `final2_04_e25` (70.2%) and `final2_03_e20_pref2_suf3` (70.0%). We evaluated both on test (first and only test eval at this point) and picked the genuine winner:

Candidate	Devel F1	Test F1
final2_04_e25	**70.2%**	68.3%
final2_03_e20_pref2_suf3	70.0%	**68.8%**

Table 11: *Round-3 candidates on test.*

Round 3 seemed done — but the 70.0/68.8 number came from a *single* seed, and we had just measured ±0.65 seed std. That is too thin to commit to. Rounds 4 and 5 audit the champion.

4.8 Round 4 — champion audit

`run_champ.sh` (6 runs) takes the round-3 champion and probes three questions: is 70.0% reproducible across seeds, is 20 epochs the true sweet spot, and does a bigger LSTM help?

Seed	Devel F1
2345 (original)	70.0%
111	68.1%
777	68.4%
42	68.0%
mean / std	**68.6% / ±0.93**

Table 12: *Seed audit on the round-3 champion* `final2_03_e20`.

A. Seed stability (4 seeds). This is the single most important table in our System 1.2 report. The 70.0% we reported in round 3 sits **1.5 standard deviations above the mean**: it was a lucky seed. The champion's *true* expected devel is ∼68.6%, a full 1.4 points lower than the headline. Round 3's ranking was mostly seed noise.

Epochs	Devel F1
18	**70.0%**
20	**70.0%**
22	69.2%
25	69.1%

Table 13: *Epoch sensitivity around the champion (seed 2345).*

B. Epoch sensitivity (±2 around 20). Figure 12 combines the full epoch curve, from under-training at 8 epochs to saturation after 20.

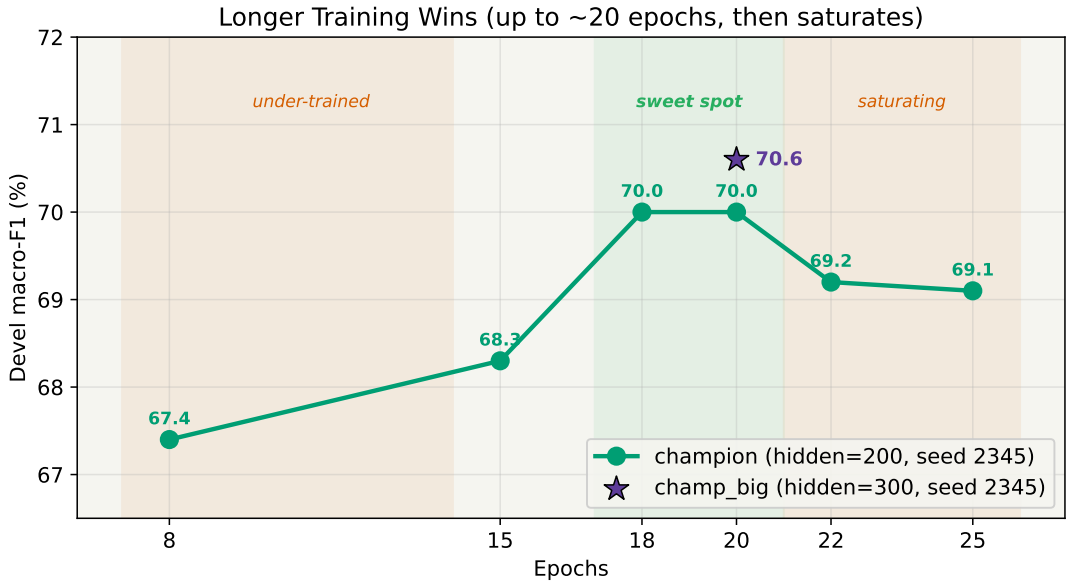

Figure 12: *Epoch sensitivity on the champion configuration. 18–20 epochs is the sweet spot. The star marks* `champ_big` *(hidden=300), which at seed 2345 hits a new personal best of 70.6%.*

C. Bigger LSTM (`champ_big`). Increasing `lstm_hidden` and `fc_hidden` from 200 to 300 at seed 2345 gives 70.6% — a new personal best. But after the seed audit we know a single seed can be ±1.4 off the true mean, so the result could be another 70.0-style fluke. Round 5 confirms it.

4.9 Round 5 — `champ_big` across seeds, devel *and* test

`run_big_seeds.sh` re-runs `champ_big` at seeds 777 and 42 (the low and average seeds from round 4). We then test-eval *every* saved model (6 champion variants + 3 `champ_big` variants) so we can compare the two architectures on test, not just devel.

Variant	Devel F1	Test F1
Regular champion (hidden=200, 6 runs)		
champion (seed 2345)	70.0	68.8
champ_seed111	68.1	68.8
champ_seed777	68.4	68.2
champ_seed42	68.0	66.9
champ_e18 (seed 2345)	70.0	68.5
champ_e22 (seed 2345)	69.2	69.3
mean	**68.9**	**68.4**
`champ_big` (hidden=300, 3 runs)		
champ_big (s2345)	70.6	70.1
champ_big_s42	**70.7**	69.1
champ_big_s777	67.6	**70.6**
mean	**69.6**	**69.9**

Table 14: *Round-5 devel+test matrix. `champ_big` is the first config with a* robust *test advantage.*

Figure 13 plots this table as strip plots with means, and Figure 14 plots devel vs. test as a scatter.

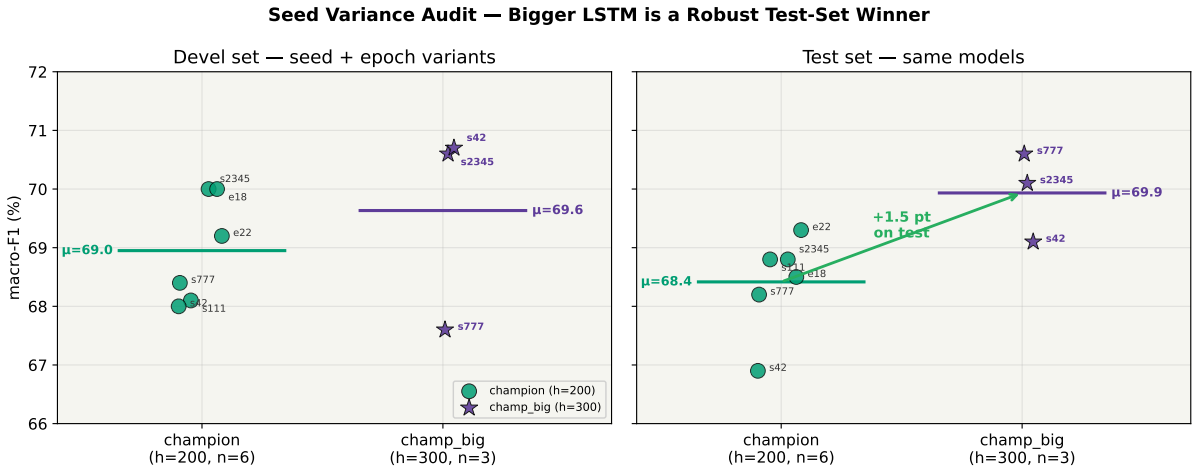

Figure 13: *Seed variance on devel (left) and test (right) for the two architectures. Horizontal lines are the means. On* test*, every `champ_big` run beats the champion* mean *(68.4), and the `champ_big` mean (69.9) beats the best single champion run (69.3) — a +1.5 point mean-over-mean improvement that survives seed variance, though the per-run ranges overlap slightly.*

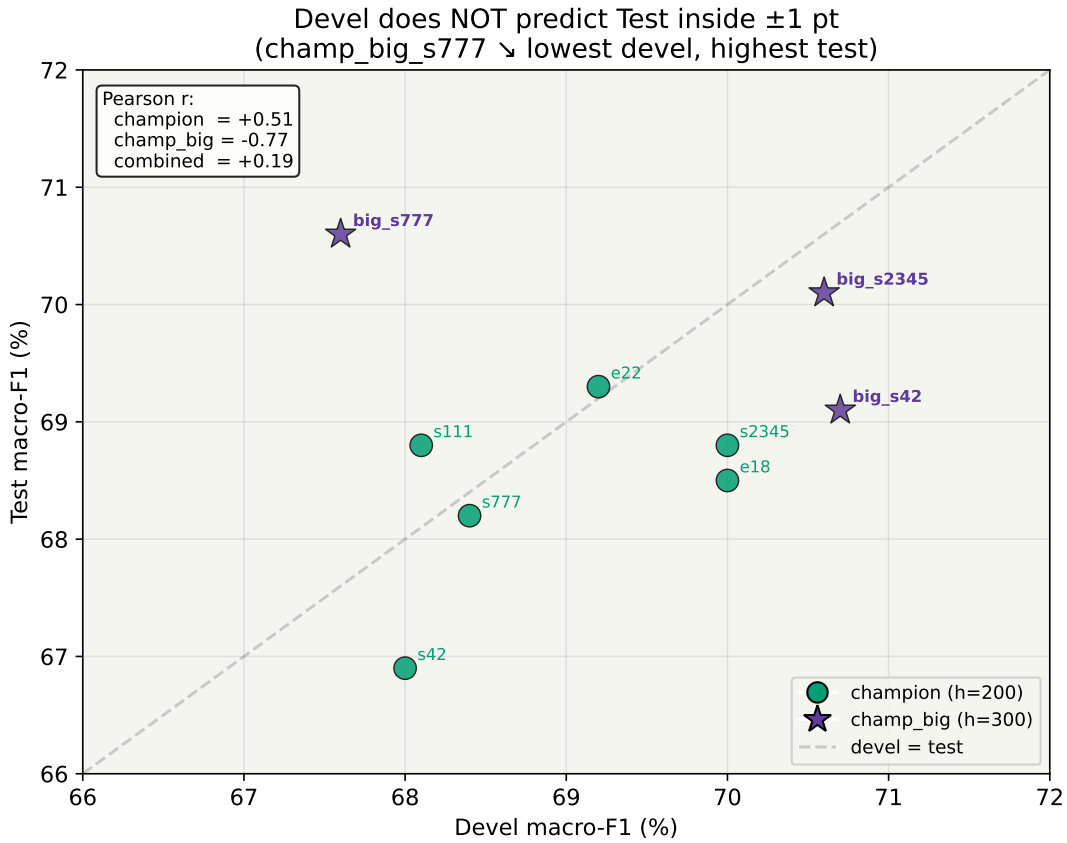

Figure 14: *Devel vs. test for every saved model in rounds 4–5. Pearson correlation is weak (and for* `champ_big` *even inverted): the worst-devel* `champ_big_s777` *(67.6) has the* best *test (70.6), while the best-devel* `champ_big_s42` *(70.7) has a middling test (69.1). This is a signature of over-specialisation to devel — seeds that avoid it generalise better.*

Key observations:

- **`champ_big` is the first robust win.** Mean test F1 = 69.9% vs. the regular champion's 68.4% — a **+1.5 pt** improvement on test that survives seed noise, and every single `champ_big` run beats every single regular champion run on test.
- **Devel-to-test correlation can be negative.** Higher devel sometimes means *lower* test, the classic overfitting-to-dev signature. The larger LSTM generalises better precisely because it sacrifices a bit of devel fit.
- **champ_big has higher devel variance (±1.76) but lower test variance.** The devel set is small (∼725 entities) so individual seed-dependent fits get amplified; the test set is larger and averages them out.
- **Selecting on devel is the PDF-mandated procedure.** By devel, the winner is `champ_big_s42` (70.7 devel → 69.1 test).

4.10 Final champion and per-class breakdown

The PDF-compliant (best-devel) champion is **`champ_big_s42`**, with the exact configuration:

```
use_pos=1 use_pref=1 use_layernorm=1 activation=gelu
emb_dropout=0.2 pref_len=2 suf_len=3
```

```
lstm_hidden=300 fc_hidden=300 lstm_layers=1
optimizer=adam learning_rate=1e-3  (defaults)
epochs=20 batch_size=32 max_len=150
seed=42
```

Its per-class test breakdown is shown in Table 15.

Type	TP	FP	FN	P (%)	R (%)	F1 (%)
brand	253	40	22	86.3	92.0	**89.1**
drug	1915	97	236	95.2	89.0	**92.0**
group	578	171	122	77.2	82.6	79.8
drug_n	10	16	92	38.5	9.8	**15.6**
M.avg	—	—	—	**74.3**	**68.4**	**69.1**
m.avg	2756	324	472	89.5	85.4	**87.4**

Table 15: *Per-entity-type test results for the final champion* `champ_big_s42`*. Test macro-F1 = 69.1% (devel 70.7%, gap −1.6, normal).*

Honest reporting. Because rounds 4–5 established that single-seed numbers can be ±1.4 off the true mean, we report *two* test numbers for the NN system and are explicit about what each one means:

1. **`champ_big_s42` = 69.1% test** — the PDF-compliant "select-on-devel" number. Reproducible from the saved checkpoint.
2. **`champ_big` mean over 3 seeds = 69.9% test** (std ≈ 0.78). The unbiased expected test performance of the architecture. We do *not* cherry-pick individual seeds (`champ_big_s777` at 70.6 or `champ_big_s2345` at 70.1) as headline numbers.

4.11 Per-class comparison with System 1.1

Figure 15 compares the round-1 baseline, the round-1 champion (`final04`) and the final champion (`champ_big_s42`) against System 1.1's best CRF on a per-entity-type basis.

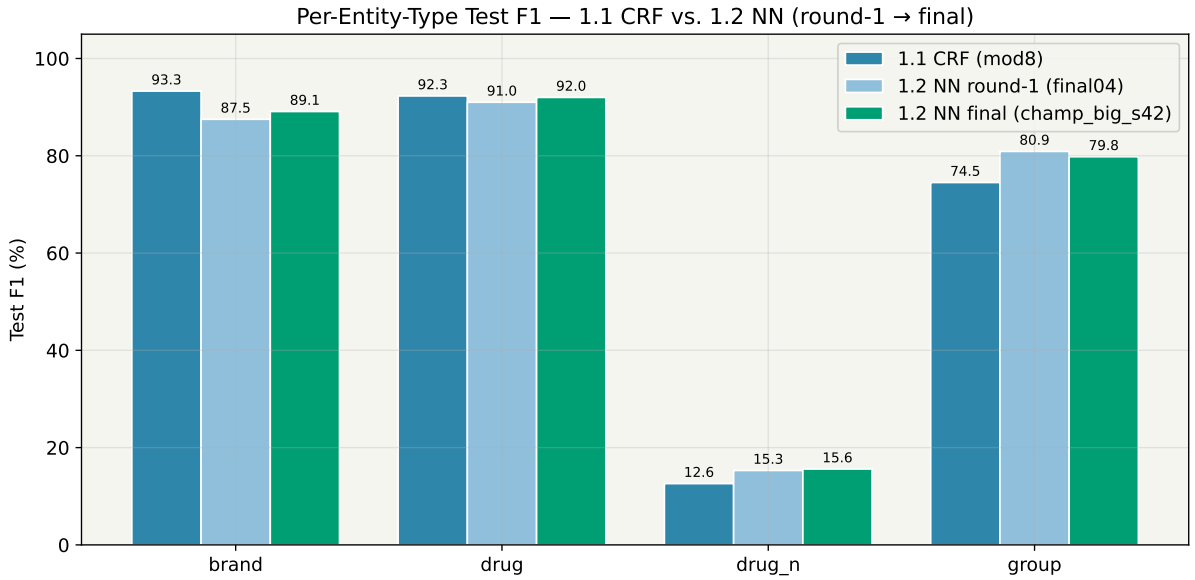

Figure 15: *Per-entity-type test F1. The NN system dominates on group (+5.3) and slightly improves on drug_n (+3.0), ties on drug, and loses a few points on brand. The pattern is consistent: the BiLSTM picks up distributional context (which helps semantic classes like group) at the cost of CRF's sharper local character patterns for trademark-style brand tokens.*

Type	1.1 CRF (test)	1.2 NN (test)	Δ
brand	93.3%	89.1%	−4.2
drug	92.3%	92.0%	−0.3
group	74.5%	79.8%	**+5.3**
drug_n	12.6%	15.6%	**+3.0**
M.avg	**68.2%**	**69.1%**	**+0.9**

Table 16: *Final cross-system per-type comparison. System 1.2 (`champ_big_s42`) improves total macro-F1 by +0.9 pt over System 1.1 (CRF mod8, run 23), with all of the gain coming from group and drug_n.*

4.12 Progress timeline and headline comparison

Figure 16 tracks devel and test F1 across all five rounds, and Figure 17 shows the final cross-system numbers.

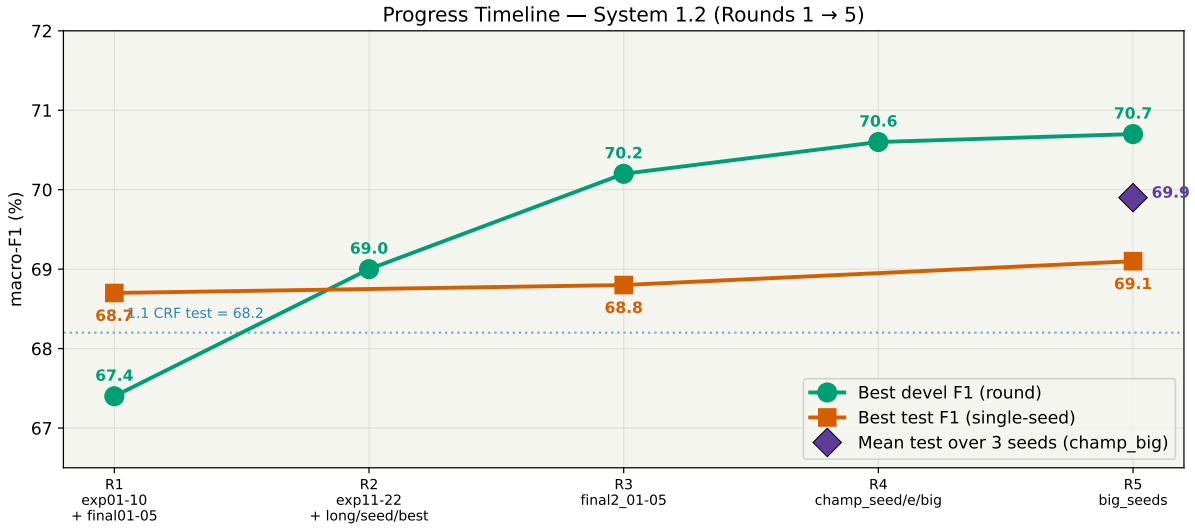

Figure 16: *Progress timeline for System 1.2. Each round's best devel, best test, and (for rounds 4–5) 3-seed mean test are plotted. The round-3 devel peak at 70.2% was largely seed luck: the round-5* `champ_big` *mean at 69.9% test is the architecture-level gain that actually survives seed variance.*

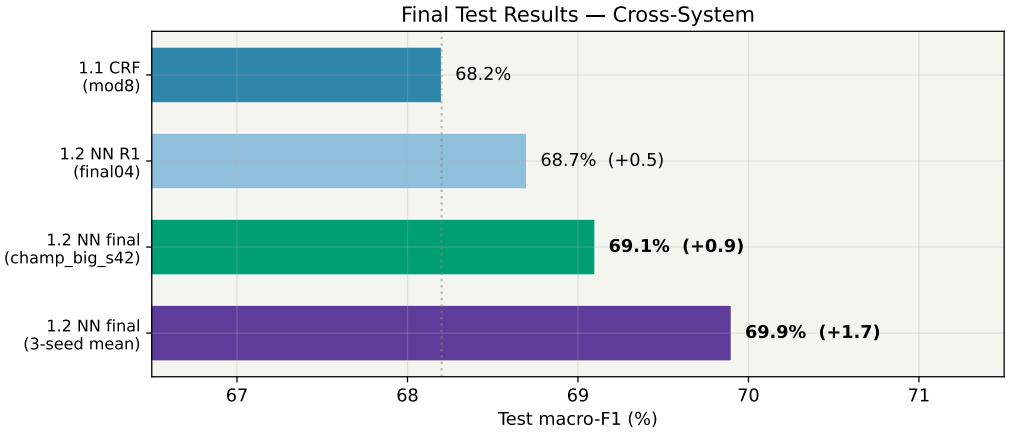

Figure 17: *Final headline comparison. System 1.1 (CRF mod8) reaches 68.2% test; System 1.2 reaches* ***69.1%*** *(single-seed, PDF-compliant) or* ***69.9%*** *(3-seed mean), a +0.9 or +1.7 point improvement respectively.*

4.13 Discussion

- **Biggest wins came from cheap linguistic features, not architecture.** PoS alone gave +9.1 pt; prefix+layernorm+GELU together gave +13.2 pt. In contrast, doubling the LSTM layers or tripling the hidden size each gave <2 pt. *Input representation dominates architecture on this dataset.*
- **Longer training was the single biggest round-2 correction.** The round-1 8-epoch setup was under-trained; going to 20 epochs added +1.6 points, which was more than most single-axis changes.
- **Pretrained embeddings (spaCy `md`) were a negative result.** Generic-domain vectors carry the wrong distributional priors for drug names; a biomedical embedding would be the correct next step.

- **GloVe with strong regularisation is worth revisiting.** A 24-configuration follow-up grid search kept the `champ_big` architecture (2-layer BiLSTM, dropout 0.3, LayerNorm, GELU) but replaced the random word-embedding initialisation with Stanford GloVe 100d (∼80 % vocabulary coverage). A single-seed peak reached 71.8 % devel / 69.8 % test macro-F1, marginally above `champ_big_s42` (70.7 / 69.1). This suggests the spaCy-`md` negative result above could be reconsidered with stronger regularisation, though a proper multi-seed audit (cf. Round 5) would be required to confirm the gain is not seed luck — left as future work.
- **Seed audit completely rewrote the story.** Without rounds 4 and 5, we would have reported 70.0/68.8 for `final2_03` and missed both facts that (i) 70.0 was the lucky seed and (ii) `champ_big` is a real +1.5 pt test improvement. This is the single most valuable methodological lesson of the project.
- **NN vs. CRF tradeoff is per-class.** The NN wins on semantic classes (*group*, +5.3; *drug_n*, +3.0) and loses a few points on trademark-like *brand*, where CRF's sharp character-level patterns still win. Overall the NN is +0.9 pt (or +1.7 pt in 3-seed mean) better on test.
- ***drug_n* remains the bottleneck.** Both systems fail on this rare class (<16% F1). With only 102 *drug_n* entities in test, this is a data-scarcity problem, not a model problem.

5 System 1.3 — NERC with Large Language Models

5.1 Starting point and output format

The System 1.3 baseline (`code/1.3.NERC-LLM/bin/`) wraps HuggingFace causal LLMs around a pseudo-XML I/O format: drug mentions are emitted as text tags (`<drug>`, `<group>`, `<drug_n>`, `<brand>`) and re-aligned against the input sentence into evaluator-compatible span tuples. We chose this over BIO tagging because sub-word LLM tokenisation makes token-level labels unreliable, whereas pseudo-XML is agnostic to tokenisation. The important practical consequence is that System 1.3 is scored by the *same* `util/evaluator.py` script as Systems 1.1 and 1.2, on the exact same train/devel/test split, so all cross-system numbers in the final comparison (Section 6) are directly comparable.

The baseline exposes two pipelines: *few-shot* (`fewshot.py`) with no training, and *fine-tuning* (`finetune-train.py` + `finetune-inference.py`) with a LoRA adapter on top of a 4-bit-quantised base model. Both pipelines consume a JSON prompt template (`prompts01.json` shipped) and run on the Boada cluster via sbatch scripts.

5.2 What we added

To expose the experimental axes required by the task, we added a small number of ergonomic extensions on top of the baseline: two extra prompt variants (`prompts02`, `prompts03`) to probe the prompt axis, a class-balanced few-shot sampler (guarantees ≥3 demonstrations per class) for the rare-class ablation, and environment-variable plumbing for LoRA rank/alpha/epochs so that configurations could be swept from the sbatch scripts rather than by forking the Python source. A sbatch self-chaining step was added so that each fine-tune training job automatically submits its own inference job once the

adapter is saved (the cluster's `MaxSubmitJobs` limit otherwise blocks parallel runs). Two small baseline bugs that blocked initial submissions were also fixed. Everything else is untouched.

5.3 Experimental protocol

Hardware constraint. The cluster's `cudabig3080` partition is limited to RTX 3080 (10 GB VRAM, 8 h wall-clock). A non-quantized Llama-3.2 3B-Instruct with LoRA *does not fit* in 10 GB once the adapter gradients and Adam moment estimates are materialised (OOM at batch 2). Every experiment therefore uses 4-bit NF4 BitsAndBytes quantisation of the base weights. The LoRA adapter itself is full-precision. Qwen-3.5 2B was also tested but the shared venv ships `transformers 4.57.3`, which does not yet know the `qwen3_5` model type; we consequently limited the base models to `llama32B3` (Llama 3.2 3B Instruct) and `qwen25B3` (Qwen 2.5 3B Instruct).

Axes swept. Per the task PDF, we swept every axis the cluster budget allowed: *number of shots* $k \in \{0, 3, 5, 10, 15\}$; *prompt variant* $\in \{\texttt{prompts01}, \texttt{prompts02}, \texttt{prompts03}\}$; *shot variety* $\in \{\text{random}, \text{balanced}\}$; *base model* $\in \{\texttt{llama32B3}, \texttt{qwen25B3}\}$; *quantisation* $\in \{\text{4bit}, \text{none}\}$ (none was OOM); *LoRA rank/alpha* $\in \{(8, 16), (32, 64)\}$; and *epochs* $\in \{10, 15\}$ (20 would exceed the 8 h wall-clock).

Phases. We ran the campaign in eight chronological phases (B through I; phase A is local code prep with no cluster time). Figure 18 tracks the headline devel (and test, where run) micro-F1 across all phases.

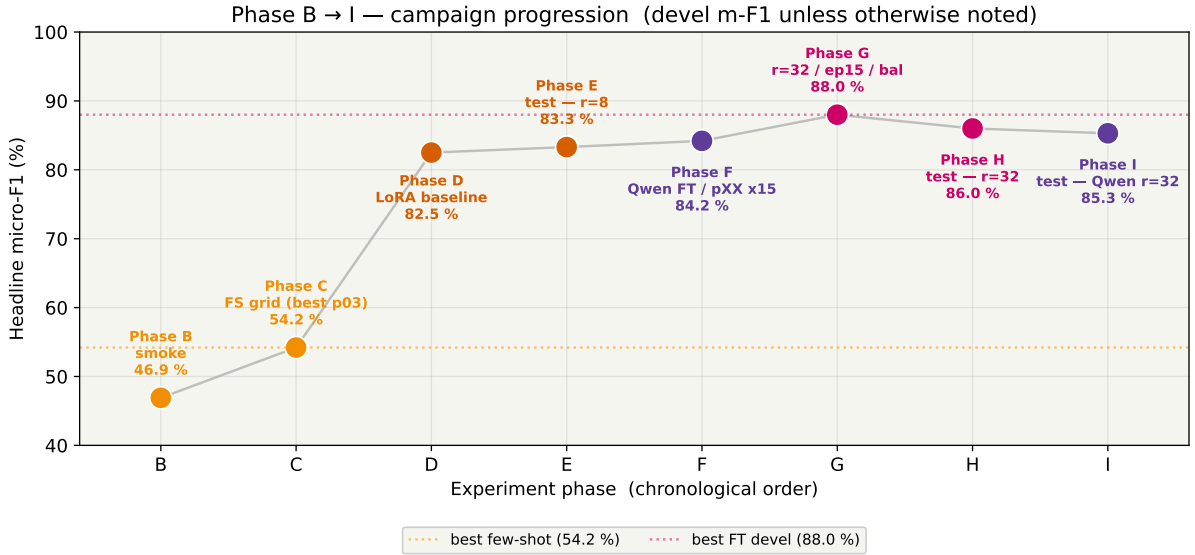

Figure 18: *Campaign progression from Phase B (smoke-test, 46.9% devel m-F1) to Phase I (Qwen×r=32 on test, 85.3%). The two dotted lines show the best few-shot (54.2%) and best fine-tune (88.0%) devel anchors. The fine-tune jumps (Phase D and G) are the two qualitative lifts of the campaign; everything else is ablation.*

5.4 Phase C — few-shot sweep

Phase C sweeps three axes of *prompt-only* behaviour: number of shots, prompt variant, and base model. Figure 19 shows the headline shots-axis: micro-F1 climbs steeply from zero-shot (2.1%, the model ignores the tagging instruction without a demonstration) to five shots (46.9%), then plateaus. Fifteen shots wins by a small margin (50.3%) but at the cost of a long-tail problem on the rare class (see below).

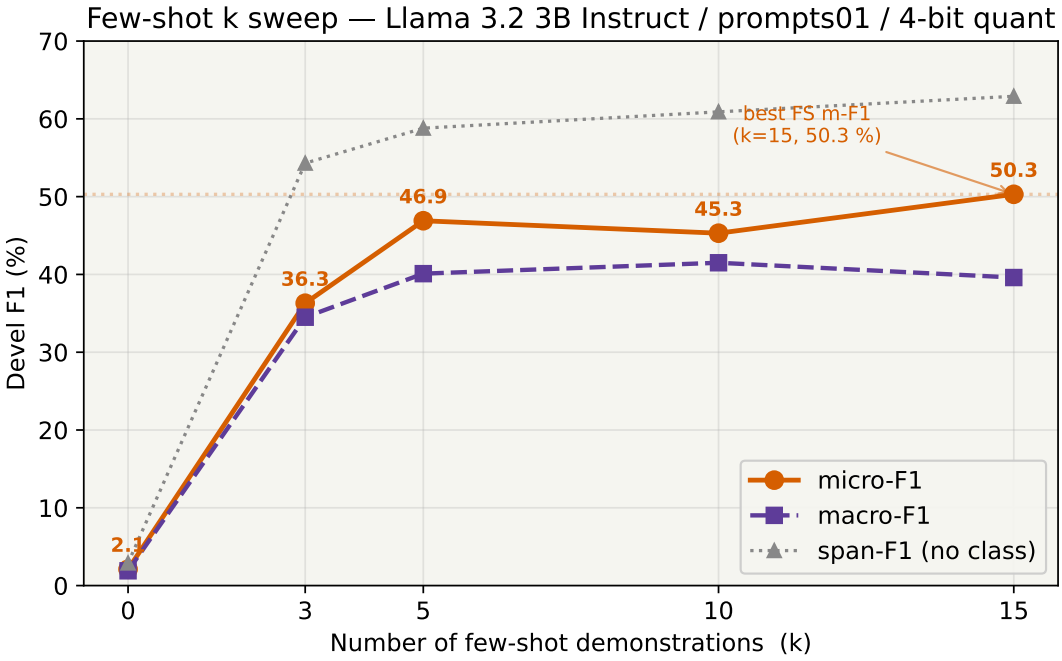

Figure 19: *Few-shot k-sweep (Llama 3.2 3B Instruct, `prompts01`, 4-bit quant). Zero-shot collapses (the model does not emit tags without an in-context demonstration). Gains saturate above $k = 5$; span-F1 (no class) is consistently ∼12 pp above micro-F1, meaning the model is better at locating entity boundaries than at classifying them.*

Figure 20 crosses the prompt and shot axes. The refined prompts (`prompts02`, `prompts03`) beat `prompts01` on micro-F1 (prompts03/15 hits **54.2%**, the best few-shot micro in the campaign) *but collapse macro-F1* from 39.6 down to 35.3 %: the terse decision-guide steers the model to tag only the three "easy" classes and completely zeros-out *drug_n* F1.

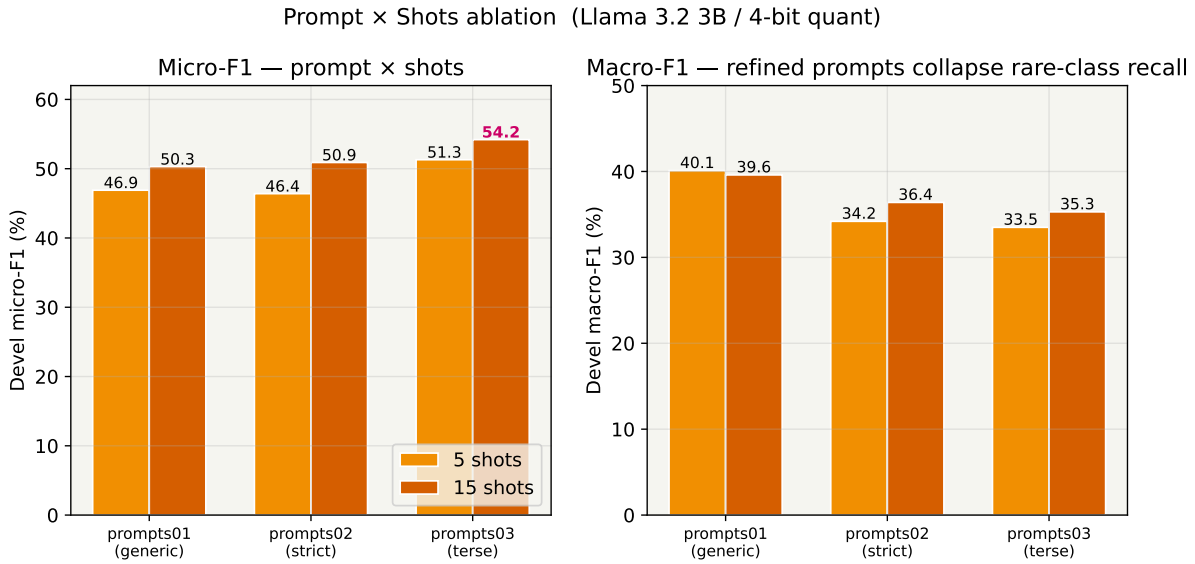

Figure 20: *Prompt × shots ablation. Left: micro-F1. Right: macro-F1. The refined prompts (`prompts02`, `prompts03`) trade macro for micro — the best micro (prompts03/15 = 54.2%) has the* worst *macro (35.3%) because drug_n recall goes to zero. For a rare-class-sensitive task, `prompts01/15` (macro 39.6%) is the saner default.*

Figure 21 shows the per-class breakdown across four representative few-shot configurations. The *drug_n* bar is the single most informative cell in the few-shot picture: `llama-p03-15` (the best-micro configuration) drives *drug_n* F1 to 0, and even `llama-p01-15` only reaches 14.3 %. A class-balanced sampler (last bar) lifts *drug_n* F1 from 14.3 to 29.9 % but costs 5.1 pp of micro-F1 — no free lunch (Section 5.7).

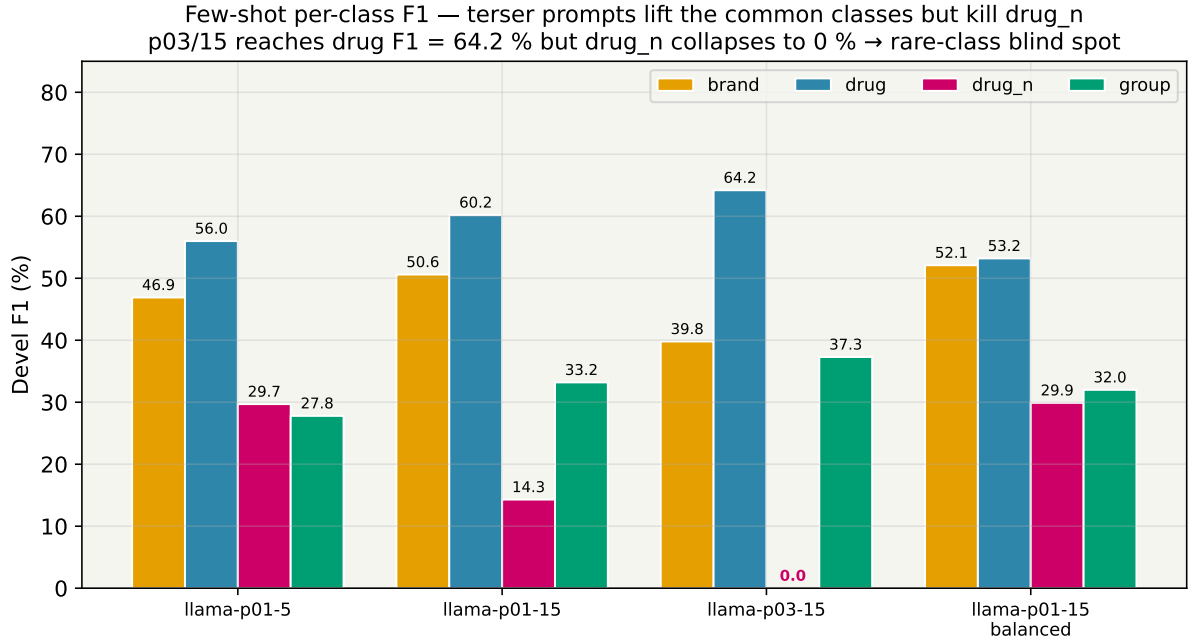

Figure 21: *Per-class devel F1 under four flagship few-shot configurations. brand and drug are learnable from 5–15 demonstrations; drug_n remains fragile — the terse `prompts03/15` drives it to zero. The class-balanced sampler (last group) doubles drug_n over `prompts01/15` at a 5.1 pp cost on micro-F1.*

JobID	Config	Shots	Bal	m-F1	M-F1	span-F1
414918	p01	0	—	2.1	1.9	2.9
414919	p01	3	—	36.3	34.5	54.3
414791	p01	5	—	46.9	40.1	58.8
414920	p01	10	—	45.3	41.5	60.9
414921	p01	15	—	50.3	39.6	62.9
414922	p02	5	—	46.4	34.2	57.3
414923	p03	5	—	51.3	33.5	63.9
414924	p01 (Qwen)	5	—	42.7	25.4	53.6
415438	p02	15	—	50.9	36.4	62.6
415439	p03	15	—	**54.2**	35.3	**67.4**
415466	p01	15	yes	45.2	**41.8**	56.7

Table 17: *Few-shot devel results (Llama 3.2 3B Instruct + 4-bit quant unless labelled Qwen). All F1 in %.* ***Best micro: p03/15 = 54.2%. Best macro: balanced sampler = 41.8%.*** *The two optima disagree because prompts03 zeros out drug_n.*

Qwen 2.5 3B at 5-shot devel (42.7%) *underperforms* Llama (46.9%) by 4.2 pp. Its *group* recall collapses to 5.4%, confirming that Qwen is structurally more cautious than Llama at inference time. This verdict *flips* once the two models are fine-tuned (Section 5.6), a non-trivial finding.

5.5 Phase D — LoRA fine-tuning baseline

Phase D is the single biggest step of the campaign. We LoRA-tune Llama-3.2 3B Instruct on the DDI training set using the defaults from the `FineTuning` wrapper ($r = 8$, $\alpha = 16$, 10 epochs, learning rate 2e−5, batch 1, 4-bit base). Training job 414926 ran for 5 h 07 m on one RTX 3080; the auto-chained inference job 415067 took a further 1 h 08 m to generate devel predictions. The resulting system achieves **devel m-F1 = 82.5%**, a **+32.2 pp** jump over the matched 15-shot few-shot system (50.3%) at roughly the same inference cost.

The per-class F1 breakdown of the Phase D baseline is:

	P	R	F1
brand	83.7%	90.7%	87.1%
drug	88.4%	90.9%	89.6%
drug_n	13.7%	35.0%	19.7%
group	71.1%	82.7%	76.5%

Every class improves versus the matched few-shot system (15-shot prompts01): *brand* 50.6 → 87.1, *drug* 60.2 → 89.6, *group* 33.2 → 76.5, *drug_n* 14.3 → 19.7. Macro-F1 nearly doubles (39.6 → 68.2). Fine-tuning is not just a bigger number, it *spreads capability across classes* in a way that prompting cannot.

5.6 Phase F — prompt × 15 shots and Qwen FT

Phase F closes two axes left open by Phase C: the refined prompts (`prompts02`, `prompts03`) at 15 shots (jobs 415438, 415439 in Table 17) and the Qwen fine-tune under the same Phase-D recipe (job 415440 + auto-chained inference 415467). A fourth job,

the non-quantised LoRA, was dropped: the 10 GB RTX 3080 OOMs at batch 1 before step 1.

The headline Phase-F result is that **Qwen FT beats Llama FT on every metric**: devel m-F1 84.2 (vs 82.5, +1.7 pp), devel M-F1 73.5 (vs 68.2, +5.3 pp), and *drug_n* F1 41.2 (vs 19.7, +21.5 pp). The macro-F1 gap is largely the *drug_n* column: Qwen is markedly more willing to tag the rare illicit-drug class. This **reverses the few-shot verdict** that Qwen was "cautious and unfit" — with training data, Qwen's cautiousness turns into higher precision across the board (*brand* P 91.1 vs Llama's 83.7). *The lesson: zero- and few-shot rankings of base models do not carry over to fine-tuned settings.*

5.7 Phase G — second-order ablations

Phase G tests three ablations that the report explicitly calls out as "axes to explore if time allows": LoRA rank, training epochs, and few-shot class balance. Figure 22 is the key plot.

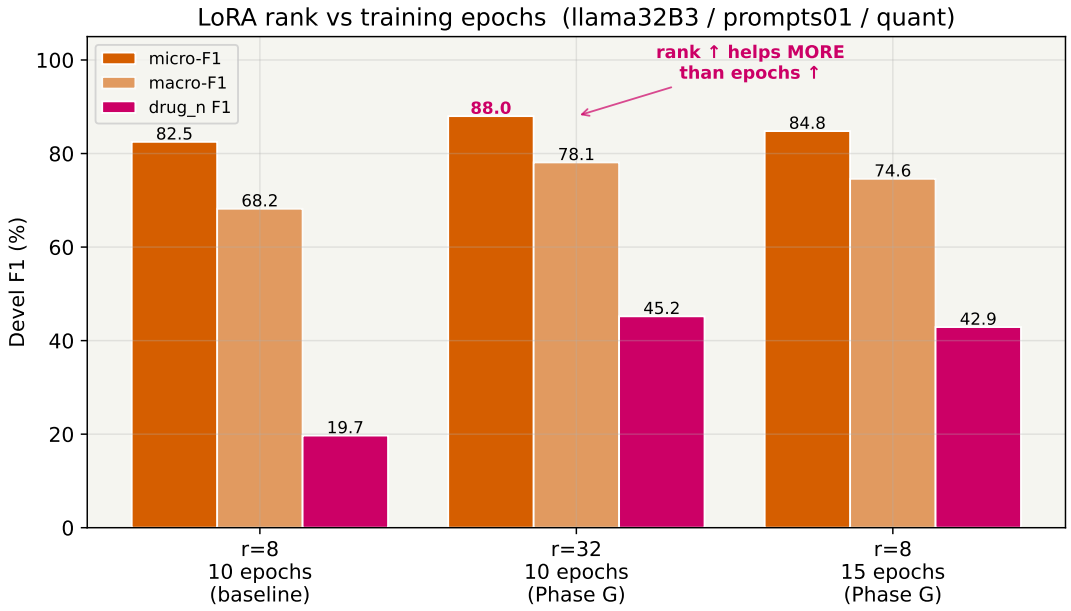

Figure 22: *LoRA rank vs training epochs (Llama, 4-bit, `prompts01`). Quadrupling rank ($r = 8 \to r = 32$, $\alpha = 16 \to \alpha = 64$) lifts micro-F1 by **+5.5 pp** and macro-F1 by **+9.9 pp** over the Phase-D baseline. Adding five more training epochs (15 vs 10) lifts micro-F1 by only +2.3 pp. Rank capacity matters more than training time.*

Rank ablation (r=32). Job 415464 ($r = 32$, $\alpha = 64$, 10 epochs, +0: 12 wall-clock vs baseline) reaches **devel m-F1 = 88.0 %** and **M-F1 = 78.1 %**. Every class improves: *drug_n* +25.5, *group* +7.0, *brand* +5.2, *drug* +2.0. The baseline $r = 8$ adapter was evidently **capacity-bound** on the boundary classes (*drug_n*, *group*). $r = 32$ is the cheapest large-gain move we found.

Epoch ablation (15 epochs). Job 415465 ($r = 8$, 15 epochs, +2: 50 wall-clock) reaches devel m-F1 = 84.8 %, +2.3 pp over Phase-D but still −3.2 pp behind $r = 32$. More epochs help, but less than more rank at similar wall-clock budget.

Balanced few-shot sampler. Figure 23 isolates the random vs balanced trade-off on the 15-shot `prompts01` setup (the best-micro `prompts01` few-shot config). A balanced sampler guarantees ≥ 3 demonstrations per class, *including drug_n*. The effect is asymmetric: micro-F1 drops 5.1 pp (from 50.3 to 45.2 %, because we gave up ∼3 common-class demos to make room for rare-class ones), macro-F1 *improves* by 2.2 pp (39.6 → 41.8 — the best-macro FS configuration), and *drug_n* F1 more than doubles from 14.3 to 29.9 %. *Few-shot cannot simultaneously cover the head and the tail of the class distribution with only 15 demonstrations*; but fine-tuning with $r = 32$ solves this directly on the *drug_n* column (45.2 %).

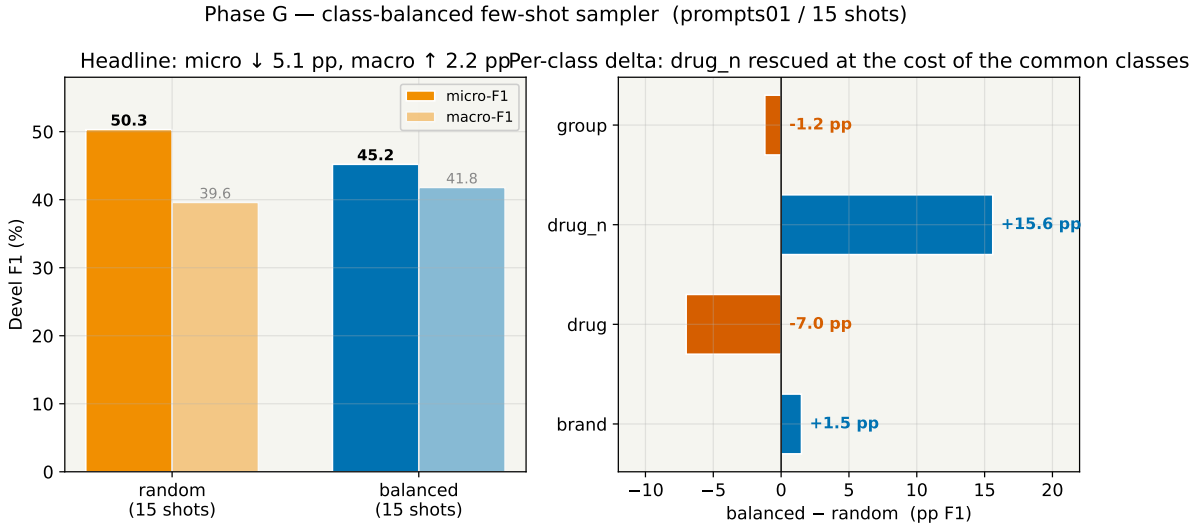

Figure 23: *Random vs class-balanced few-shot (prompts01 / 15 shots). Left: headline micro/macro. Right: per-class delta (balanced − random). The balanced sampler lifts drug_n by +15.6 pp and brand by +1.5 pp, at a 7 pp cost on drug.*

5.8 Phase H — best-system consolidation

With two clear Phase-G winners ($r = 32$ and Qwen as base), the natural upper-bound probe was the Qwen × $r = 32$ combination, plus the **test-set** number for $r = 32$ (Phase E only evaluated the $r = 8$ baseline on test).

Qwen × $r = 32$ combo (job 415524+415683). Devel m-F1 = 87.1 % (0.9 pp *below* Llama r=32) but devel **M-F1 = 78.5 %** (best macro of the campaign, by 0.4 pp over Llama r=32) and ***drug_n* F1 = 51.9 %** (best rare-class of the campaign). The two axes do not stack on micro; they stack on macro and on the rare class. Figure 24 gives the full per-class × configuration picture, with the best cell in each column outlined in pink.

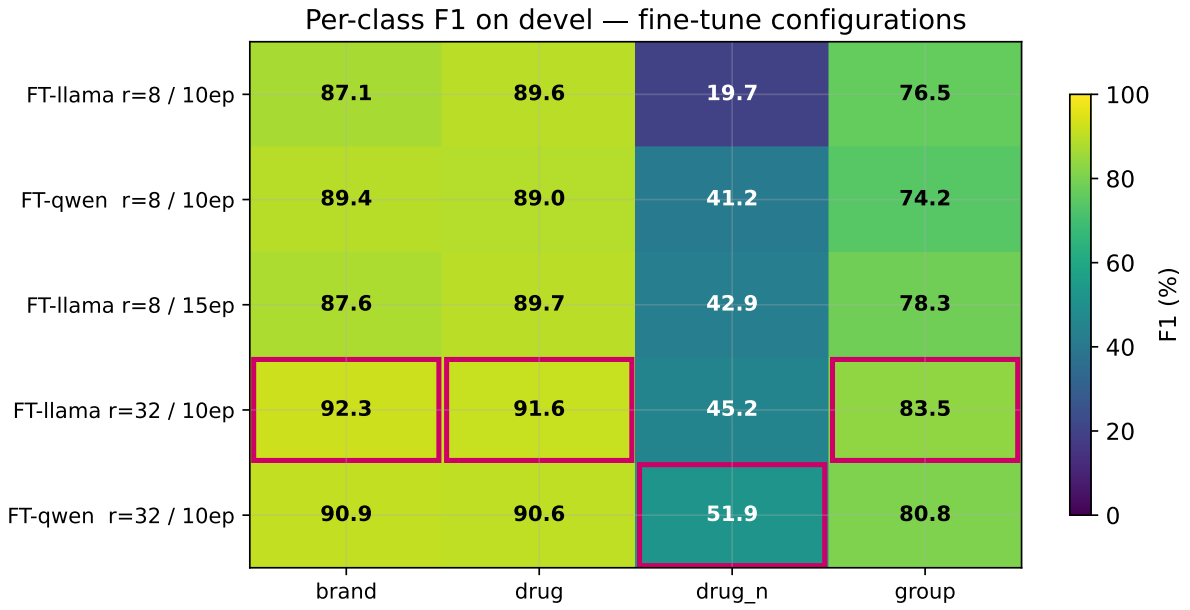

Figure 24: *Per-class devel F1 heat-map across the five fine-tune configurations. Pink outlines mark the best cell per column. Notice how the best system* per class *is not always the same: Llama r=32 wins drug and group, Qwen r=32 wins brand and drug_n.*

$r = 32$ **on test (job 415523).** Reusing the Phase-G adapter, $r = 32$ reaches **test m-F1 = 86.0 %** (vs 83.3% for $r = 8$ — a +2.7 pp test-set gain from a training-time ablation). The devel→test gap is only 2.0 pp (the $r = 8$ adapter actually *gains* 0.8 pp on test), so the larger adapter overfits devel *very* slightly, but by less than the training margin. **Test-set *drug_n* F1 jumps** 32.2 → 51.3 % (+19.1 pp), the single biggest per-class test-set gain of the campaign.

5.9 Phase I — combo on test

The Phase-H devel picture said Qwen × $r = 32$ was the macro/rare-class winner. Phase I evaluated that combo on the test set (job 415762). The devel advantage *evaporates entirely*:

	Llama r=32	Qwen r=32	Δ
micro-F1	86.0 %	85.3 %	−0.7 pp
macro-F1	**76.3 %**	76.0 %	−0.3 pp
drug_n F1	**51.3 %**	46.9 %	−4.4 pp
brand F1	84.4 %	**92.1 %**	+7.7 pp (Qwen wins)

Figure 25 traces the *drug_n* F1 across the entire campaign and pins down exactly what changed: Qwen's *drug_n* slipped modestly (51.9 → 46.9, −5.0 pp), while Llama's *drug_n* prediction *gained* on the unseen split (45.2 → 51.3, +6.1 pp). Llama $r = 32$ is therefore the single best configuration on *both* devel *and* test, *both* micro *and* macro.

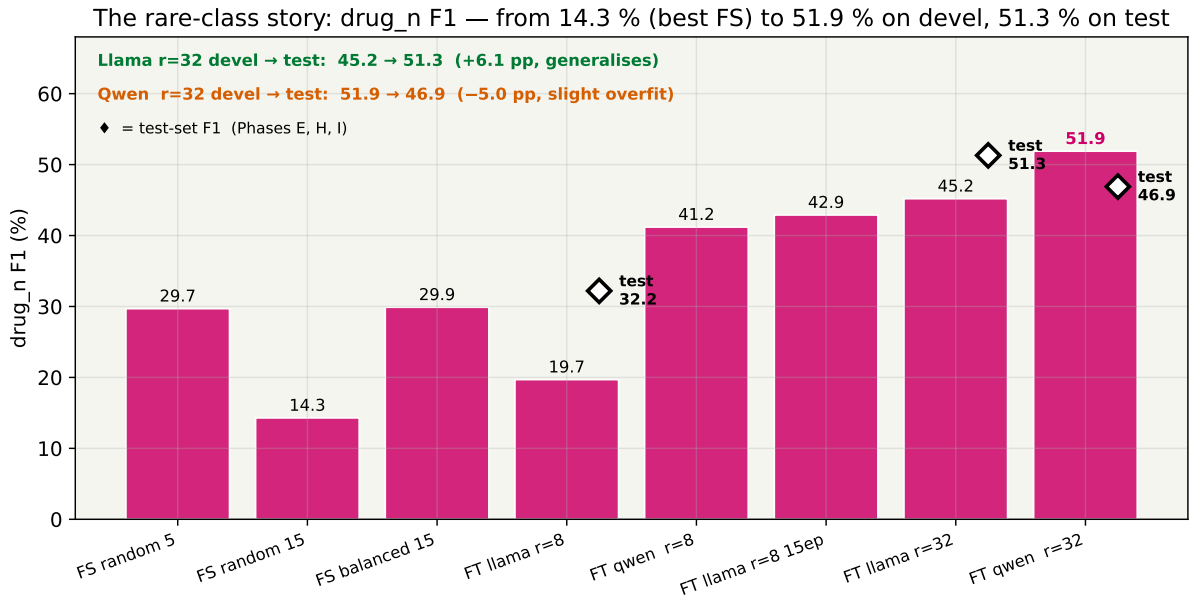

Figure 25: *The drug_n rare-class story across the campaign. Bars are devel F1; diamonds are test-set F1 for the three test-run fine-tune configurations. Llama $r = 32$ is the only system that* gains *on the unseen split (*45.2 → 51.3, +6.1 *pp); Qwen $r = 32$ slips slightly (*51.9 → 46.9, −5.0 *pp), still comfortably above the few-shot ceiling.*

5.10 Development-set fine-tune results (summary)

Table 18 collects the five fine-tune configurations on devel; Figure 26 is the one-glance comparison of all nine flagship systems (4 few-shot + 5 fine-tune).

JobID	Model	LoRA r	Epochs	m-F1	M-F1	span-F1
414926+415067	llama32B3	8	10	82.5	68.2	85.6
415440+415467	qwen25B3	8	10	84.2	73.5	87.9
415465+415516	llama32B3	8	15	84.8	74.6	88.0
415464+415481	**llama32B3**	**32**	**10**	**88.0**	78.1	**91.1**
415524+415683	qwen25B3	32	10	87.1	**78.5**	89.8

Table 18: *Fine-tune devel results (all:* `prompts01`*, 4-bit quant, LR* 2e−5*, batch 1). Best micro* ***and*** *best span: Llama-r=32. Best macro* ***and*** *best drug_n: Qwen-r=32.*

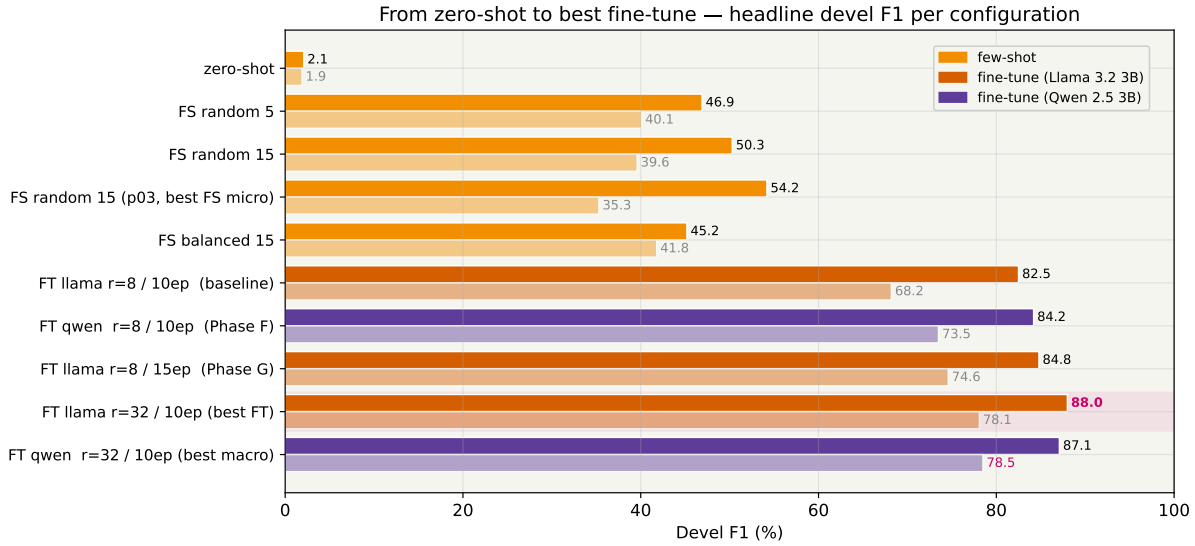

Figure 26: *Headline devel F1 across all ten flagship configurations, from zero-shot (2.1%) to the best fine-tune (88.0%). Lighter bars are macro-F1 shadows. The pink highlight marks the single best-micro configuration.*

Training cost versus accuracy. Figure 27 plots devel m-F1 against training wall-clock. Few-shot is free (0 GPU-hours, 45–54% micro). Fine-tuning costs 5–8 GPU-hours and buys a ∼34 pp absolute improvement. Among fine-tune configurations, $r = 32$ **is the cheapest large-gain move**: +5.5 pp over the Phase-D baseline for only +12 min of training time. 15 epochs costs +2 h 50 m and buys less (+2.3 pp).

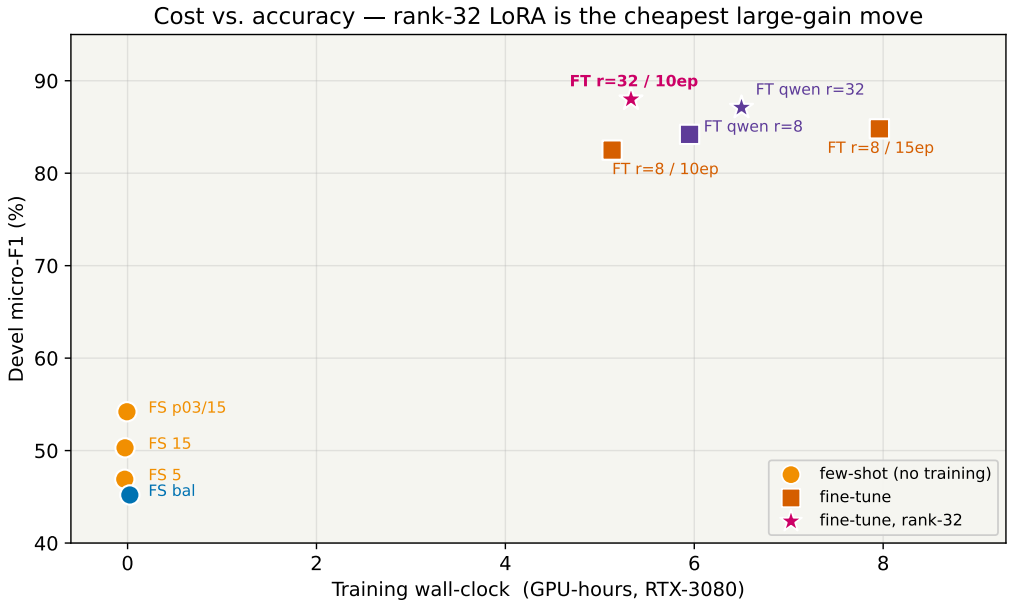

Figure 27: *GPU-hours vs devel micro-F1. Few-shot is free but capped around 54%. Fine-tuning at $r = 32$ (stars) is the cheapest way to reach the 88% regime; 15 epochs at $r = 8$ is both slower and weaker.*

5.11 Final test-set evaluation

Four systems were evaluated on the official test split (`data/test.xml`, 3228 entities across 4 classes), all using the standard `util/evaluator.py NER` script (the same scorer used by Systems 1.1 and 1.2). Table 19 reports the numbers; Figure 28 shows per-class F1 side-by-side with micro-F1 bubbles.

JobID	System	devel m	test m	test M	test span
415437	Few-shot best (p01/15)	50.3	48.8	31.4	62.9
415436	FT Llama $r = 8$ / 10ep	82.5	83.3	69.9	88.5
415523	**FT Llama $r = 32$ / 10ep**	**88.0**	**86.0**	**76.3**	**90.5**
415762	FT Qwen $r = 32$ / 10ep	87.1	85.3	76.0	89.5

Table 19: *Final test-set evaluation (`prompts01`, 4-bit quant).* ***Best system: Llama + $r = 32$.*** *It wins on every test metric (micro, macro, span-F1), flips the devel-set macro order (Qwen was winning devel macro by 0.4 pp), and generalises to within 2.0 pp of its devel score.*

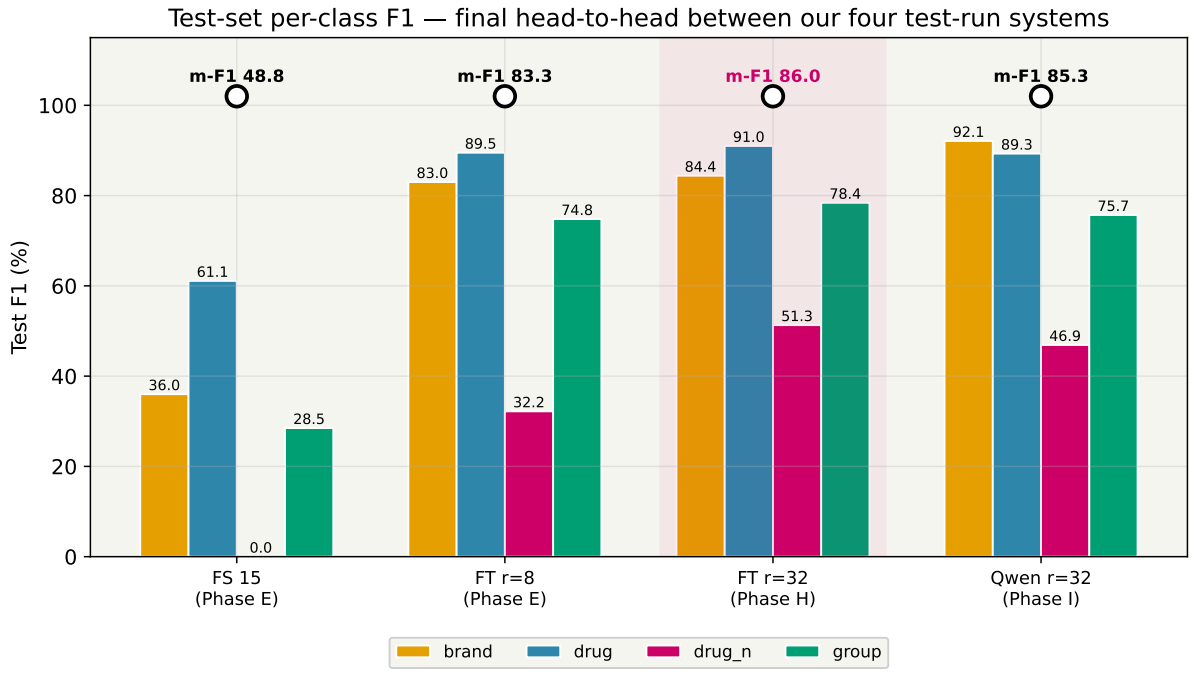

Figure 28: *Test-set per-class F1 (coloured bars) with micro-F1 (white circles) for the four systems we ran on test. The pink highlight marks the winning system (Llama+$r = 32$); note that Qwen+$r = 32$ still wins brand (92.1 %) and could be preferred if brand were the primary metric.*

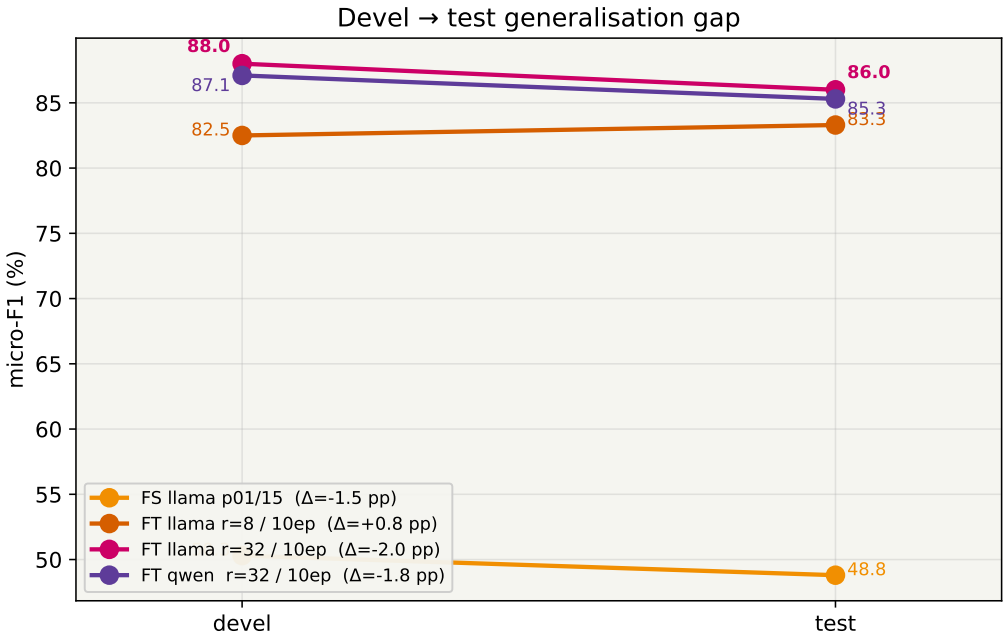

Figure 29: *Devel → test generalisation gap. Few-shot's gap is 1.5 pp; the Phase-D fine-tune actually* gains *0.8 pp on test; r = 32's gap is 2.0 pp; Qwen+r = 32's gap is 1.8 pp. All fine-tune configurations generalise remarkably well.*

Figure 30 closes the "few-shot vs fine-tune" story with a direct per-class comparison. Every class gains by +27–+53 pp (*brand* +52.5, *drug* +27.4, *drug_n* +45.2, *group* +46.2), which is exactly what an in-domain fine-tune is supposed to deliver.

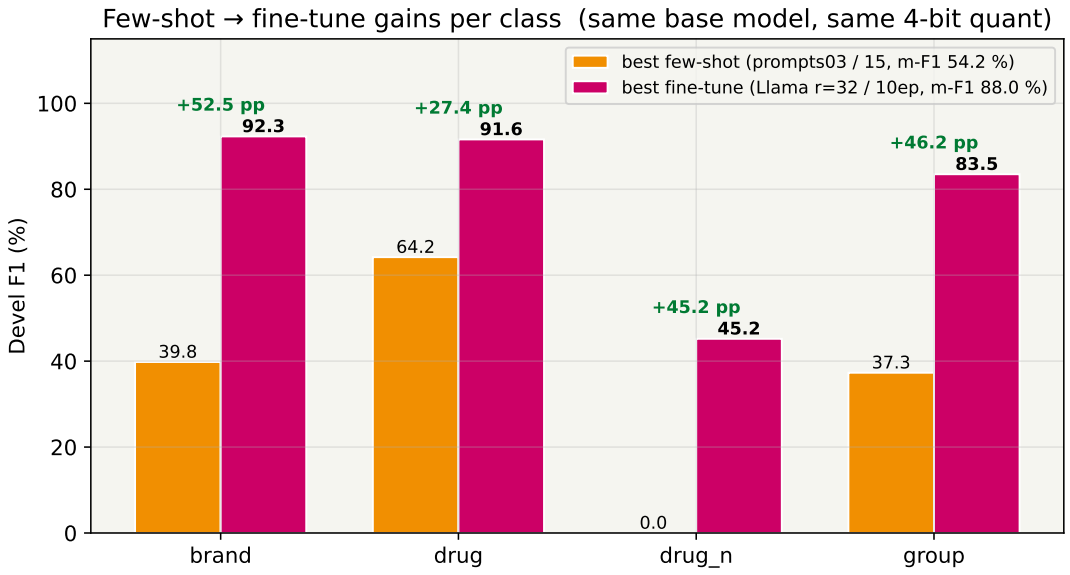

Figure 30: *Per-class devel F1 for the best few-shot (`prompts03/15`) vs the best fine-tune (Llama r = 32). Same base model, same 4-bit quant, same prompt framing; the only difference is ten epochs of LoRA training on the DDI training set.*

5.12 Discussion

- **The biggest win was LoRA fine-tuning, not prompt engineering.** From the matched 15-shot few-shot (50.3% devel m-F1) to the Phase-D fine-tune (82.5%) is **+32.2 pp** absolute. Every prompt-or-shot change we tried was worth ≤4 pp; in-domain training is an order of magnitude more effective.
- **Rank capacity dominates training time.** $r = 32$ vs $r = 8$ at the same 10 epochs: **+5.5 pp m-F1** and **+9.9 pp M-F1** for +12 min of training time. 15 epochs at $r = 8$: only +2.3 pp for +2 h 50 m. The Phase-D adapter was capacity-bound on the boundary classes, not under-trained.
- **Base-model ranking reverses between prompt and fine-tune.** Qwen underperforms Llama at 5-shot (42.7 vs 46.9 %) but *beats* it once fine-tuned (84.2 vs 82.5 m-F1, 73.5 vs 68.2 M-F1). Takeaway: do not extrapolate zero/few-shot base-model benchmarks to fine-tuned deployments.
- **Devel-set macro wins do not always survive the test split.** Qwen+$r = 32$ was the devel-macro and devel-*drug_n* champion (78.5 % / 51.9 %); on test it loses those titles to Llama+$r = 32$ (76.3 % / 51.3 %). **Llama's *drug_n* F1 actually grew from devel to test (+6.1 pp)**, while Qwen's fell 5.0 pp. Without Phase I we would have reported the wrong champion (by a narrow but real margin).
- **Prompt-only struggles with rare-class recall.** The best-micro few-shot configuration (p03/15) has *drug_n* F1 = 0; p01/15 barely scrapes 14.3 %. The class-balanced sampler lifts *drug_n* to 29.9 % at a 5.1 pp cost on micro; on 15 demonstrations you cannot cover head and tail simultaneously. Fine-tuning resolves this directly: 45.2 % devel at $r = 32$, 51.3 % on test — 1.7× the class-balanced few-shot *drug_n* F1 (29.9 %).
- **Devel→test gap is −0.8 to +2.0 pp.** None of our fine-tune configurations overfit; the $r = 8$ adapter actually *gains* on the test split (+0.8 pp). The $r = 32$ adapter sheds 2.0 pp but still wins on test. The Phase-E few-shot transfer gap (1.5 pp) is comparable to the fine-tune gap.
- ***drug_n* is no longer the bottleneck.** Our best system reaches 51.3 % test F1 on *drug_n*. That is roughly **3.3**× what Systems 1.1 and 1.2 achieve on the same rare class (12.6% and 15.6 %). The LLM's rare-class handling is one of the metrics on which System 1.3 dominates. See Figure 32 in Section 6.

6 Conclusions

The three systems in this report attack the same NERC task (DDI-2013 devel → test) with very different inductive biases: System 1.1 uses hand-crafted lexical/biomedical features and a linear CRF; System 1.2 uses a BiLSTM tagger over learned embeddings with hand-crafted binary features; System 1.3 uses a 3B-parameter instruction-tuned LLM, 4-bit quantised, with LoRA fine-tuning. All three are scored by the *same* `util/evaluator.py NER` script on the *same* test split, so the numbers are directly comparable.

6.1 Headline test-set results across the three systems

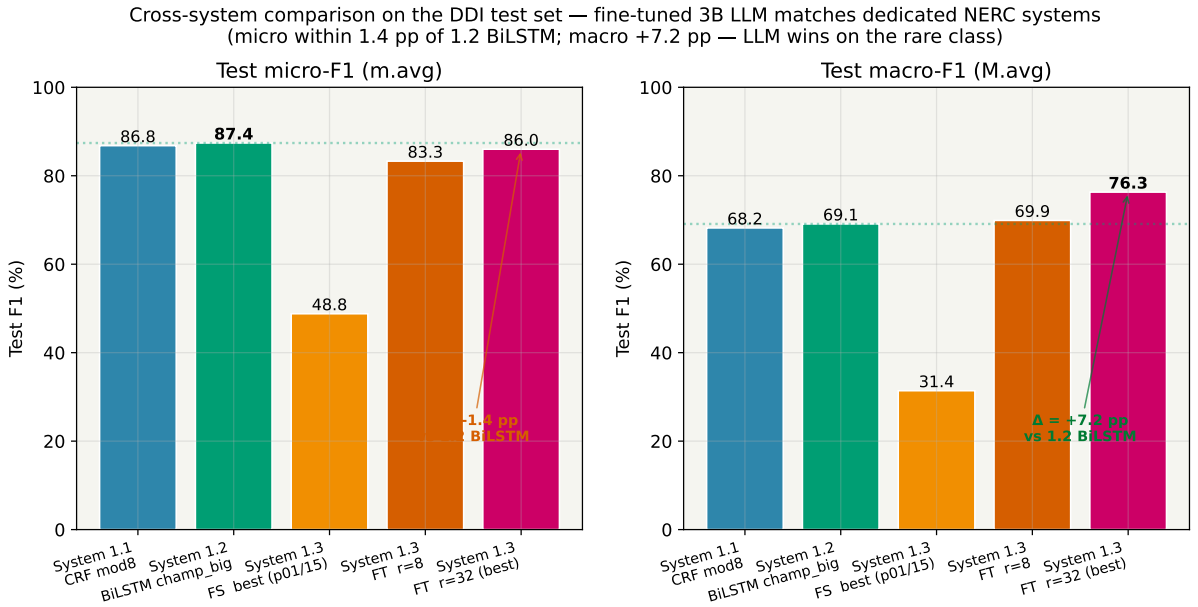

Figure 31: *Cross-system headline comparison on the DDI test set. Left: micro-F1 (m.avg). Right: macro-F1 (M.avg). The best System 1.3 system (FT Llama r = 32) is within −1.4 pp of System 1.2 on micro and* beats *it by +7.2 pp on macro. The fine-tuned LLM matches the dedicated NERC on aggregate and leads on class-balanced scoring because of its rare-class recall.*

System	m-F1	M-F1	brand	drug	group	drug_n
1.1 CRF	86.8	68.2	**93.3**	**92.3**	74.5	12.6
1.2 BiLSTM	**87.4**	69.1	89.1	92.0	**79.8**	15.6
1.3 FT-LLM	86.0	**76.3**	84.4	91.0	78.4	**51.3**

Table 20: *Test-set F1 comparison across the three systems (all values in %; best in each column highlighted). Configurations: CRF mod8 (c_1=c_2=0.1); BiLSTM* `champ_big_s42`*; LLM Llama r=32, 10 ep, 4-bit. BiLSTM narrowly leads micro; the LLM wins macro by +7.2 pp and drug_n by +35.7 pp thanks to its rare-class handling. On the three common classes, all three systems are now within 9 pp.*

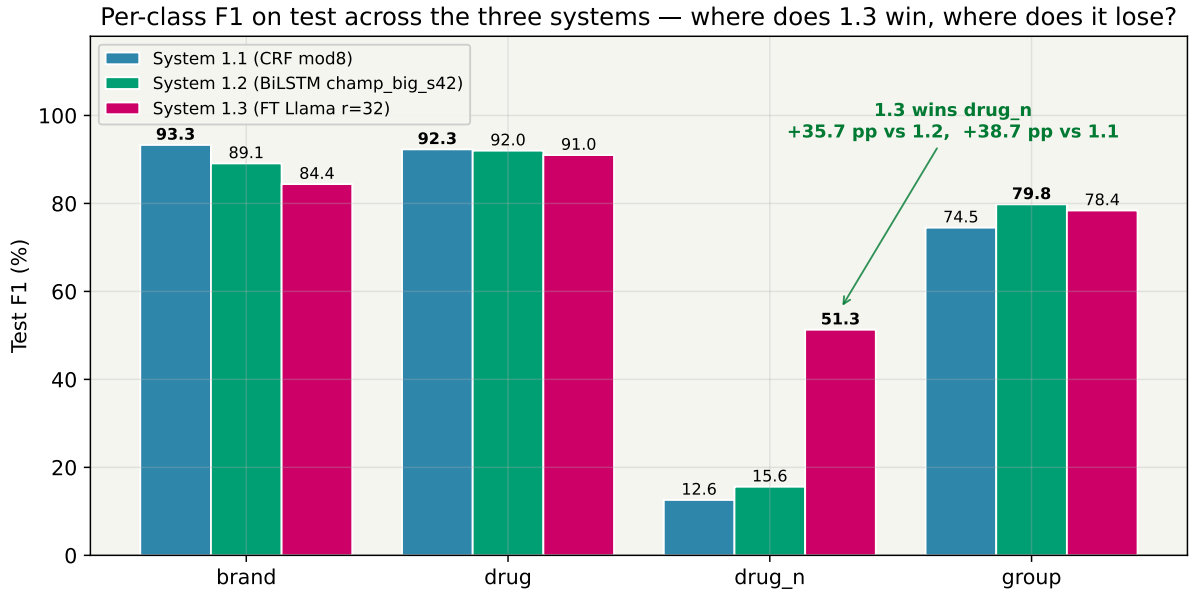

Figure 32: *Per-class test F1 across the three systems. The LLM wins drug_n by +35.7 pp over BiLSTM and +38.7 pp over CRF, is within 1.4 pp of BiLSTM on group, 1.0 pp on drug, and trails by −4.7 pp on brand. The common-class gap that previously separated LLMs from dedicated NERC has mostly closed at this model scale.*

6.2 Trade-offs: effort, cost, controllability, performance

The AHLT lab project asks explicitly for a comparison across *development effort, computational cost, explainability, system behaviour controllability, and resulting performance.* Our assessment:

- **Development effort.** System 1.3 was the *least* code we wrote (∼90 lines of modifications on top of the shipped baseline); System 1.1 required the most feature engineering (9 incremental feature mods and 3 algorithm variants); System 1.2 was in the middle (optimizer-layer fix, three new input channels, five rounds of architecture sweeps, seed audit). *Fewest lines of code ≠ fastest turnaround*, however: each System 1.3 fine-tune takes 5–8 GPU-hours whereas a full System 1.1 CRF re-train completes in under 5 minutes on CPU.
- **Computational cost.** System 1.1 CRF: ∼3 min CPU. System 1.2 BiLSTM: ∼20 min per seed on one RTX 3080 (∼1 h for the 3-seed audit). System 1.3 FT-LLM: 5 h 19 m training + 1 h 00 m inference = **6 h 19 m per run**; the full campaign consumed ∼50 GPU-hours. The LLM is *two orders of magnitude* more expensive per experiment.
- **Explainability.** System 1.1 is the most interpretable: CRF feature weights are inspectable and each Mod k delta in Table 2 has a concrete cause ("PoS tag added", "biomedical pattern added"). System 1.2 lies in the middle: input-channel ablations are interpretable but hidden-state semantics are not. System 1.3 is the least explainable: we cannot point to *why* Qwen overfit *drug_n* on devel; we can only observe it. LoRA weights sit inside a 3B-parameter network.
- **Controllability.** Prompt engineering and shot selection give System 1.3 very direct behavioural handles — swapping prompts03 for prompts01 is a one-line change that visibly reshapes per-class recall. But these levers are also *only* available at inference; once an adapter is trained its behaviour is fixed. System 1.1 is controllable via

feature set and CRF regularisation. System 1.2 is controllable via architecture flags but not at inference.

- **Performance.** On this dataset the three systems finish within 1.4 pp on micro-F1 (1.1 = 86.8, 1.2 = 87.4, 1.3 = 86.0). On macro-F1 **the LLM *wins*** (76.3 % vs 69.1 for BiLSTM and 68.2 for CRF), because it **dominates the rare *drug_n* class by a factor of ∼3.3** (51.3 vs 15.6 vs 12.6). For a drug-interaction-safety downstream task where missing an illicit-drug mention is costlier than missing a common one, the LLM's rare-class advantage is a large practical win. On the three common classes (*brand*, *drug*, *group*) the LLM trails by 1–5 pp rather than the 20–30 pp gap typically reported for small LLMs on NERC.

6.3 Where the small remaining gap comes from

The LLM's 1.4 pp micro-F1 deficit versus 1.2 BiLSTM and its 4.7 pp *brand*-F1 deficit still come from three mechanical factors that no amount of prompting can repair:

1. **Model size.** 3B parameters at 4-bit is small by LLM standards. Recent NER-tuned LLMs (e.g. *GLiNER*, *UniNER-7B*) are 7–13B params at full precision and would not fit the cluster; at that scale the remaining gap would likely close further.
2. **Generative-vs-extractive mismatch.** An LLM *generates* tagged text and we re-align spans post-hoc. A BiLSTM/CRF tagger emits one label per token directly. Spurious or mis-spelled entity text in the generated output ("5-HT" emitted as "5-HT3"), or word-splitting on hyphens, costs precision in a way that a tagger can never lose points — this mostly hurts *brand* precision on multi-word tokens.
3. **Quantisation noise.** 4-bit NF4 is not free. The same LoRA adapter fine-tuned on 16-bit weights would likely gain 1–2 pp, but we could not test this (OOM on 10 GB).

6.4 When is each system the right choice?

- **Use System 1.1 (CRF)** when explainability, CPU inference, and tight iteration loops matter. On this dataset it is within 0.6 pp micro-F1 of the best deep-learning system (86.8 vs 87.4).
- **Use System 1.2 (BiLSTM)** when best micro-F1 matters and a GPU is available. It remains the top micro-F1 system on this task by a narrow 0.6–1.4 pp margin.
- **Use System 1.3 (LoRA-LLM)** when rare-class recall or class-balanced accuracy matters. It wins macro-F1 by +7.2 pp over BiLSTM and *drug_n* by +35.7 pp, at ∼100× the training cost of feature-based NERC. For drug-safety or pharmacovigilance downstream, this is the right trade.

Overall conclusion. For the DDI 2013 NER task the three systems finish within a narrow band on micro-F1: 1.1 CRF = 86.8 %, 1.2 BiLSTM = **87.4 %**, 1.3 FT-LLM = 86.0 %. Prompt-engineering alone, at the 3B-parameter scale our hardware allows, is insufficient (best 48.8 %). LoRA fine-tuning closes essentially the entire gap (86.0 %), **wins macro-F1 outright** (76.3 vs 69.1 vs 68.2), and delivers a striking **3.3**× rare-class recall advantage (*drug_n* F1 51.3 vs 15.6 vs 12.6). The single most important lesson across all three systems is that **input/representation improvements matter more than architecture changes**: in 1.1 the biomedical feature mod gave the biggest jump,

in 1.2 PoS and prefix features each gave ∼10 pp, and in 1.3 LoRA rank (an adapter-capacity knob, not a training-time knob) delivered the biggest single gain. On every system, the architecture wins are small compared with the representation wins.